

IncidentFlow

Build It Yourself

A complete, illustrated walkthrough of a production incident-management platform — every screen, every file, every command, and every bug that had to be fixed.

Written for someone who can open a project in VS Code and nothing more. No Laravel, React, Docker or SQL assumed.

Project directory: `D:\IncidentFlow`

Contents

How to read this book	1
I Understanding the system	3
1 What IncidentFlow actually is	4
1.1 The problem	4
1.2 Who uses it	4
1.3 The shape of an incident	4
1.3.1 Severity: how bad is it	5
1.3.2 Status: where is it in its life	5
1.3.3 The transitions are a rule, not a suggestion	5
1.4 Your first look	6
2 The architecture	8
2.1 Nine programs, not one	8
2.2 What each one is for	9
2.3 Following one request all the way through	9
2.4 Why not one program?	10
2.5 The security boundary	10
3 Every file, and why	11
3.1 The top level	11
3.2 Inside api/ — where the rules live	11
3.2.1 One controller per action, not one per resource	12
3.2.2 Where the real logic lives	13
3.3 Inside web/ — what you look at	13
3.4 Inside realtime/ — the live connection	14
3.5 Everything else	14
3.6 A rule of thumb for finding anything	14
II The guided tour	15
4 Sign in	16
4.1 What is deliberately missing	16
4.2 The two halves of the screen	17
4.3 The password field, and why it has an eye	17
4.4 What happens when you press Sign in	18
4.5 The error message is deliberately unhelpful	19
4.5.1 A disabled account is treated differently	20
4.5.2 Every error has the same shape	20
4.6 What sign-in actually returns	20
4.7 Check your understanding	21

5	The incidents list	22
5.1	Reading the screen	22
5.2	The button that is not always there	23
5.3	Where <code>can()</code> comes from	24
5.4	Every badge is an enum	24
5.5	The colour rule this whole design follows	24
5.6	Filtering, and why the URL changes	24
5.7	Check your understanding	25
6	The incident detail page and its timeline	26
6.1	The timeline is the product	26
6.2	Append-only, enforced three times	27
6.2.1	Why three layers and not one	28
6.3	Changing an incident	28
6.4	The page updates while you watch it	28
6.4.1	Why the browser cannot just use its normal token	29
6.5	Check your understanding	29
7	Services, metrics and postmortems	30
7.1	Services: what can break	30
7.2	Metrics: are we getting better or worse?	31
7.2.1	The numbers are precomputed, not counted on the fly	31
7.3	Postmortems: what did we learn?	32
7.3.1	Postmortems are editable — and that is not a contradiction	32
7.4	Two screens you will meet by accident	33
7.4.1	Registering an organisation	33
7.4.2	The page that is not there	34
7.5	Check your understanding	34
8	Permissions, seen rather than described	35
8.1	One page, three people	35
8.2	Where the difference is defined	37
8.2.1	Read the shape, not just the contents	38
8.2.2	Permissions are an enum, not strings	38
8.3	Now the same picture from the other side	38
8.4	The full grid, for reference	39
8.5	Try it yourself	39
9	The operator surfaces	40
9.1	Admin: the audit log	40
9.2	Horizon: watching the queue	41
9.3	Mailpit: catching email that must not escape	42
9.3.1	Watch the whole chain in one action	42
9.3.2	Try the same thing at <code>sev3</code>	43
9.4	Check your understanding	43
III	Running it	44
10	From nothing to running	45
10.1	What you need installed	45

10.2	The three commands that start everything	45
10.3	Shorter names for the same commands	46
10.4	Which command runs where	46
10.5	How the nine containers find each other	47
10.5.1	The complete connection map	47
10.6	Watching it work	47
10.6.1	Getting inside a container	47
10.7	Stopping, and the difference that matters	48
10.8	Running the tests	48
IV	Building it from nothing	49
11	The order, and why it is not arbitrary	50
11.1	The principle	50
11.2	Stage 1: the schema	50
11.3	Stage 2: enums before models	51
11.4	Stage 3: authentication, and the shape of the key	51
11.5	Stage 5: the logic, before any HTTP	51
11.6	Stage 9: Docker last, not first	52
11.7	Stage 10: what CI actually protects	52
V	Bugs, and how they were found	53
12	Bugs in the application	54
12.1	The button that broke every test	54
12.2	The manual that would have lied	54
13	Bugs in the backend	56
13.1	The one that only failed on PostgreSQL	56
13.2	The test that proved nothing	56
13.3	The header that was documented and never sent	57
13.4	The failure that turned a success into a 500	58
14	Bugs between the services	59
14.1	The dashboard that returned 200 and showed nothing	59
14.1.1	The fix that did nothing, and why	60
14.1.2	The lesson	61
14.2	The 502 that appeared while nothing changed	61
14.3	The health check that lied	61
14.4	Three smaller ones, quickly	62
14.4.1	A clean exit treated as a failure	62
14.4.2	Workers permanently unhealthy	62
14.4.3	The second ‘up’ that destroyed the stack	62
15	Bugs that only appear when deploying	63
15.1	The cookie that is never sent	63
15.2	Three containers, three different signing keys	63
15.3	A hostname that resolves and still cannot be reached	64
15.4	A message that was a count, not a diagnosis	64
15.5	Two mistakes of mine, for completeness	65

15.5.1	The tool that reported success without running	65
15.5.2	The document that destroyed itself	65

How to read this book

This book has one goal: that you could rebuild IncidentFlow *by hand*, without help, having read it.

That is a higher bar than “understand the code”. It means you need to know not only what each file contains, but why it exists, what would break without it, which command creates it, and in what order the pieces have to be built.

What is in here

- Part I Understanding.** What the system does, the shape of it, and where every file lives. Read this once, slowly. Nothing later makes sense without it.
- Part II The guided tour.** Every screen, as a picture, next to the code that produces it and the file path it lives at. This is the bulk of the book.
- Part III Running it.** Every command, every connection, every port. Written for someone who has only ever opened a folder in VS Code.
- Part IV Building it from nothing.** The order in which the pieces have to be created, and why that order is forced.
- Part V Bugs.** Real failures from building this system — the symptom, the wrong guess, the evidence, the fix. This is the part that teaches judgement.

How the pages are laid out

Three things repeat throughout, and recognising them will make the book faster to read.

FILE `api/app/Models/Incident.php`

```
// A grey bar like the one above always precedes code, and always
// names the exact file the code came from, relative to the project root.
```

A blue box explains *why* something is the way it is. The code shows what happens; these boxes explain the decision behind it.

Careful. An amber box is a mistake that is easy to make and unpleasant to diagnose. Read these even if you skim everything else.

You should see: A green box tells you what you should observe on your own machine if you followed along correctly. If you see something different, stop — do not continue until it matches.

This broke. A red box is a real failure that happened while building this system. Part V covers them in full, but they appear inline where the relevant code is introduced, so you meet the bug in the place it lives.

The one rule

Type the commands. Do not copy and paste them, and do not read past a green box whose result you have not seen on your own screen.

The difference between reading about a system and being able to build one is almost entirely made up of the small failures you hit while typing — a wrong directory, a missing container, a typo in a hostname. Skipping them feels faster and teaches nothing.

Part I

Understanding the system

What this part covers

Three chapters, and they are the ones to read slowly. The first explains what the system does and the rules it enforces. The second shows the nine programs it is built from and follows a single click all the way through them. The third maps every directory, so that for the rest of the book you always know where a file lives before you are shown what is in it.

Nothing after this point assumes you remember the details — but it does assume you have seen them once.

What IncidentFlow actually is

1.1 The problem

When something breaks in production — a payment service starts failing, a database runs out of disk — several things must happen at once. Someone has to be told. Someone has to take charge. Everyone else needs to see what is being done without interrupting the person doing it. And afterwards, someone has to be able to reconstruct exactly what happened and when.

That last part is the hard one. Chat threads get edited. Memories disagree. The engineer who fixed it at 3am writes the summary a week later.

IncidentFlow's single organising idea is that **the timeline is append-only**. Nothing written can be edited or deleted — not by an administrator, not by the person who wrote it, not by the API. If the record says the incident was acknowledged at 02:14, that is what happened. Almost every design decision in this book follows from that one commitment.

1.2 Who uses it

Five roles, deliberately few. Each can do strictly more than the one above it.

Role	What it is for
viewer	Can read everything, change nothing. Support staff, managers, anyone who needs to know without being involved.
reporter	Can raise an incident, and comment. Cannot drive one.
responder	Works incidents: acknowledges, mitigates, resolves.
incident_commander	Owns an incident. Assigns people, changes severity, closes it out.
administrator	Manages the organisation: members, services, and the audit log.

You will meet this table again in Chapter 8, where the same list appears as *screenshots of one page seen by different people*. That is the clearest way to understand a permission system: not as a table, but as buttons that are missing.

1.3 The shape of an incident

Every incident has a **severity** — how bad it is — and a **status** — where it is in its life.

1.3.1 Severity: how bad is it

Value	Name	Meaning
sev1	Critical	Everything is down, or money is being lost right now. Wakes people up.
sev2	High	A major feature is broken for many users.
sev3	Moderate	Degraded, or broken for a few.
sev4	Low	Cosmetic or minor; fix it in normal hours.

Severity is not decoration. `sev1` and `sev2` send email to every responder in the organisation; `sev3` and `sev4` do not. One enum value decides whether a hundred phones buzz at 3am — which is why changing severity is a permission a reporter does not have.

1.3.2 Status: where is it in its life

Value	Name	Meaning
open	Open	Reported. Nobody has picked it up yet.
acknowledged	Acknowledged	Someone is actively working on it.
mitigated	Mitigated	Users are no longer affected, but the underlying cause is still there.
resolved	Resolved	Actually fixed.
closed	Closed	Reviewed and finished. Terminal.

1.3.3 The transitions are a rule, not a suggestion

You cannot move an incident from any status to any other. The permitted moves are written down in exactly one place, and the API refuses everything else.

FILE `api/app/Enums/IncidentStatus.php`

```
public function allowedTransitions(): array
{
    return match ($this) {
        // A trivial incident can go straight to resolved without ceremony.
        self::Open      => [self::Acknowledged, self::Resolved],
        self::Acknowledged => [self::Mitigated, self::Resolved],
        // Mitigation that does not hold sends the incident back to active work.
        self::Mitigated  => [self::Resolved, self::Acknowledged],
        // Resolved is reversible: an incident that recurs was never resolved.
        self::Resolved   => [self::Closed, self::Acknowledged],
        // Closed is terminal. Post-closure findings belong in the postmortem.
        self::Closed     => [],
    };
}
```

Read that carefully, because three of those five lines encode a real opinion about how incidents work.

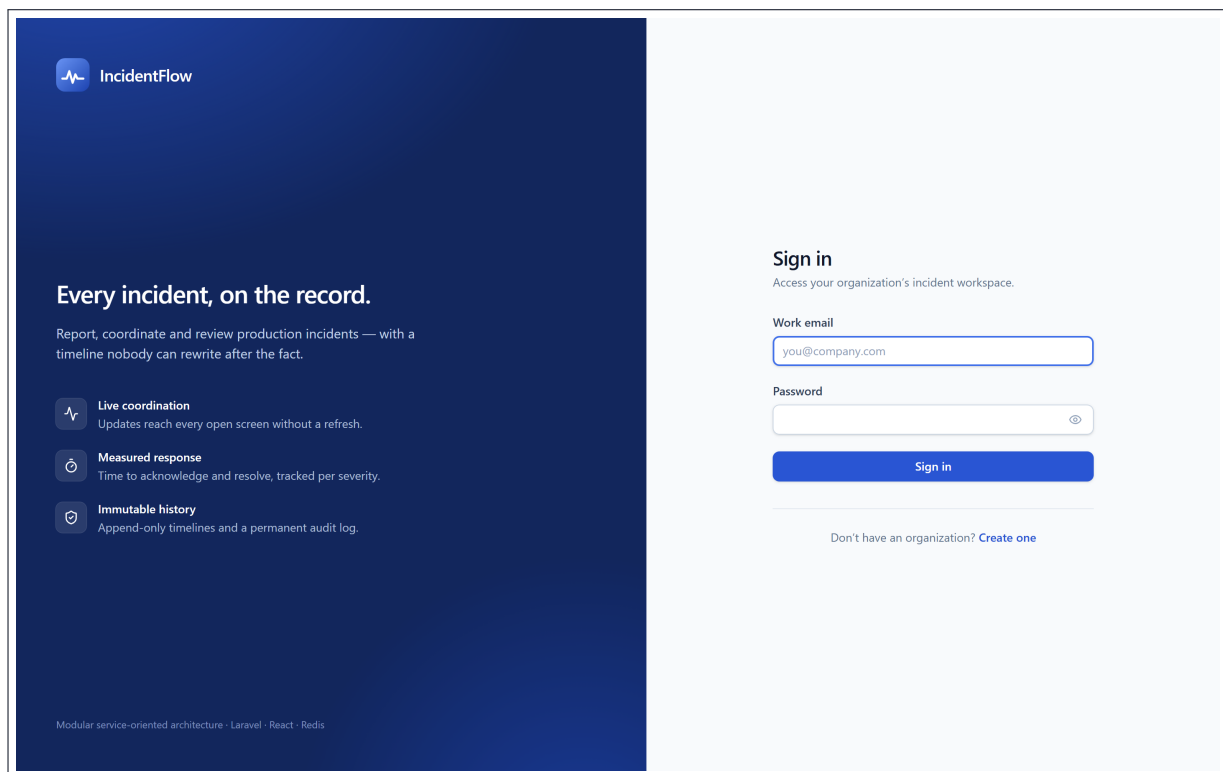
- **Open can jump straight to Resolved.** Forcing every trivial incident through an acknowledgement step produces a system people work around.
- **Mitigated can go back to Acknowledged.** A workaround that stops holding is not progress, and the timeline should say so.

- **Closed goes nowhere.** Once closed, the record is finished. Anything learned later belongs in the postmortem, which is a separate document — not a quiet edit to history.

Careful. This rule lives in the *enum*, not in the controller and not in the React app. The frontend also hides buttons for moves that are not allowed — but that is a courtesy to the user, not a security control. Anyone can send any HTTP request they like. If the rule lived only in the browser, it would not be a rule at all. You will see this pattern throughout: **the UI prevents mistakes, the API prevents abuse**, and only one of them is trusted.

1.4 Your first look

Here is the sign-in screen — the whole system’s front door.



The sign-in screen at <http://localhost:8080>. Chapter 4 takes this page apart line by line.

And here is what a signed-in incident commander sees: everything currently happening.

The screenshot shows the IncidentFlow web application interface. At the top, there's a header with the IncidentFlow logo, the company name 'Northwind Logistics', a 'Live' status indicator, the user 'Daniel Okoye', and a 'Sign out' link. Below the header is a navigation bar with tabs for 'Incidents', 'Services', 'Metrics', and 'Postmortems'. The main content area is titled 'Incidents' and shows '17 matching this filter'. There are buttons for 'Export CSV' and 'Report incident'. A search bar and filter controls (All statuses, All severities, Active only) are present. The table below lists 17 incidents with columns for Reference, Severity, Status, Title, Service, Opened, and Running / resolved in.

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	57m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 30m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

The incidents list, signed in as Daniel Okoye (incident commander). Seventeen incidents, seeded so the system has a realistic history to show.

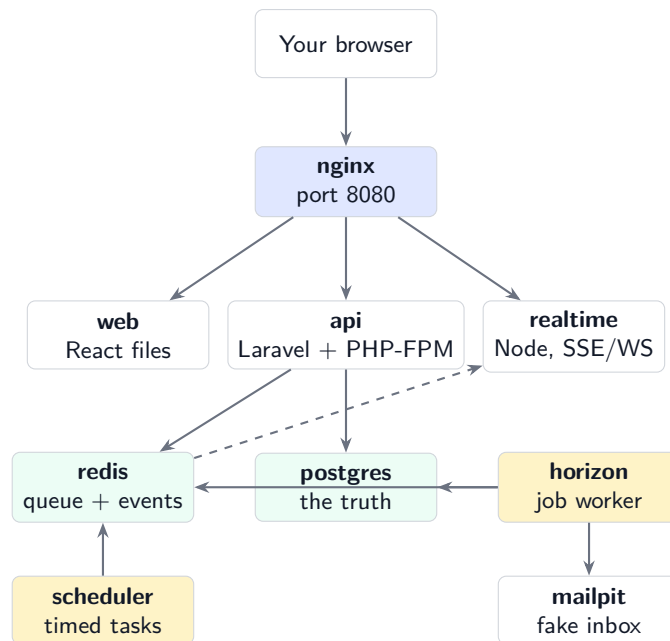
Do not worry about details yet — Part II gives each screen a full chapter. For now notice only this: every coloured badge on that page is one of the enum values from the tables above. The pictures and the code are the same thing.

The architecture

2.1 Nine programs, not one

When you run IncidentFlow, nine separate programs start. That number surprises people. A student project is usually one program; this is nine, and every one of them earns its place.

Here is the whole system on one page. Read the arrows as “talks to”.



The nine containers. Solid arrows are direct connections; the dashed arrow is events flowing from Laravel to the realtime tier through Redis. Only nginx is reachable from outside.

2.2 What each one is for

Container	Built from	Its job
<code>nginx</code>	nginx 1.27	The front door. Everything from your browser arrives here and is routed onward. Nothing else is exposed.
<code>web</code>	React + Vite	Serves the compiled JavaScript and CSS. It has no logic — it asks the API for everything.
<code>api</code>	PHP 8.3 + Laravel	All the rules. Who may do what, what a valid transition is, what gets written.
<code>realtime</code>	Node.js 22	Holds long-lived connections open so screens update without a refresh.
<code>postgres</code>	PostgreSQL 16	Stores everything. The single source of truth.
<code>redis</code>	Redis 7	Two jobs: carries live events from Laravel to realtime, and holds the queue of pending work.
<code>horizon</code>	same as api	Runs queued jobs — chiefly sending email — so the web request does not wait for them.
<code>scheduler</code>	same as api	Runs timed tasks, like sweeping notifications that were never delivered.
<code>mailpit</code>	mailpit	A fake inbox for development, so test email never reaches a real person.

Notice that `api`, `horizon` and `scheduler` are built from *the same image*. One Dockerfile, three containers, three different start commands. Without Docker you would install PHP once and run three processes; here each gets its own isolated process, so a memory leak in the queue worker cannot take the API down with it.

2.3 Following one request all the way through

Abstract diagrams are easy to nod at and hard to remember. So follow one real action: **a responder clicks “Acknowledge” on an incident.**

1. The browser sends `POST /api/v1/incidents/42/status` to `localhost:8080`. That is **nginx**.
2. `nginx` sees the path starts with `/api/`, so it hands the request to **api** over FastCGI — not HTTP. (FastCGI is the protocol web servers use to talk to PHP.)
3. Laravel checks the JWT in the `Authorization` header: who is this?
4. It checks the *policy*: may a responder change this incident’s status?
5. It checks the *enum*: is `open` to `acknowledged` a legal move?
6. Inside one database transaction it writes the new status to **postgres**, appends a timeline event, and creates notification rows.
7. The transaction commits. Only then does it push jobs onto **redis** and publish an event.
8. **horizon** picks the job off Redis and sends email through **mailpit**.
9. **realtime** is subscribed to Redis. It receives the event and pushes it down every open connection.
10. Every other open browser updates. Nobody pressed refresh.

Careful. Step 7 says “the transaction commits, *only then*” for a reason. If the job were queued before the commit, Horizon — a separate process — could pick it up and look for an incident that has not been written yet. It would find nothing and fail. This is one of the classic distributed-system bugs, and the ordering in that step is the entire defence against it.

2.4 Why not one program?

You could build all of this as a single Laravel application. Many people would. Three things here genuinely cannot be done that way.

Long-lived connections

PHP-FPM assigns a worker process per request. A connection held open for an hour would occupy a worker for an hour. Twenty users watching a dashboard would consume the whole pool and the API would stop answering anything at all. Node handles thousands of idle connections on one process, because that is what an event loop is good at. Hence a separate `realtime` service.

Slow work

Sending an email takes as long as the mail server takes. If that happened inside the request, the responder’s browser would spin while SMTP negotiated. So the API writes a row, returns immediately, and `horizon` does the sending afterwards.

Timed work

Something must run every minute regardless of whether anyone is using the system. A web request cannot do that, because there might not be one. Hence `scheduler`.

This is a *modular service-oriented* architecture, not microservices. The distinction matters: there is one database and one deployable unit. The services are split along the lines where the *runtime model* differs — request/response, long-lived connection, background work — not along business domains. Splitting by domain is what produces microservices, and with them distributed transactions, which this system deliberately does not have.

2.5 The security boundary

One rule, and it explains most of the configuration you will read later: **only nginx is reachable from outside.**

Service	Reachable from your machine?	How
nginx	yes	localhost:8080
mailpit	yes (development only)	localhost:8025
postgres	development only	localhost:5432
redis	development only	localhost:6379
api	no	only nginx, over the internal network
realtime	no	only nginx
web	no	only nginx
horizon	no	has no network port at all
scheduler	no	has no network port at all

In production those “development only” rows are closed too. You will see exactly how in Part III, in `docker-compose.prod.yml`.

Every file, and why

You cannot hold 174 PHP files in your head, and you do not need to. What you need is a reliable answer to one question: *if I want to change X, which folder do I open?* This chapter gives you that.

3.1 The top level

```
D:\IncidentFlow\  
|  
+-- api/           the Laravel backend  (all the rules live here)  
+-- web/           the React frontend  (what you look at)  
+-- realtime/      the Node service    (live updates)  
+-- infra/nginx/   the front door's configuration  
+-- scripts/       operational scripts (database backup)  
+-- deploy/        deployment runbooks  
+-- docs/          this book, and the screenshots in it  
|  
+-- docker-compose.yml      development: the nine containers  
+-- docker-compose.prod.yml  production overlay (no bind mounts, no open ports)  
+-- docker-compose.tls.yml   adds HTTPS for a single-server deployment  
+-- Makefile                 short names for long commands  
+-- README.md                the short version of this book
```

Three top-level folders — `api`, `web`, `realtime` — are the three programs *we* wrote. Everything else is either configuration that wires them together or documentation. If you remember only one thing from this chapter, remember those three.

3.2 Inside `api/` — where the rules live

This is the largest and most important part of the system. Laravel puts almost everything under `app/`.

Folder	Files	What is in it
app/Models	14	One class per database table. <code>Incident</code> , <code>User</code> , <code>AuditLog</code> .
app/Http/Controllers	19	The entry point for each HTTP request.
app/Http/Resources	11	Turns a model into the JSON the API returns.
app/Http/Middleware	6	Runs before a request reaches a controller.
app/Http/Requests	—	Validates incoming data before anything touches it.
app/Enums	9	Fixed sets of values — statuses, severities, roles.
app/Policies	4	Who is allowed to do what.
app/Services	12	The actual business logic, grouped by subject.
app/Jobs	1	Work that happens in the background.
app/Console/Commands	3	Things you can run by hand or on a schedule.
app/Exceptions	6	Named failures, and the one place errors are formatted.

And outside `app/`:

Folder	What is in it
database/migrations	8 files that create the tables. Run in order, they build the schema from nothing.
database/seeder	Fills a fresh database with realistic demo data.
routes/api.php	The map from URL to controller. Start here when tracing anything.
config/	Settings, all reading from environment variables.
tests/	13 test files, 114 tests.
docker/	The container's own files: entrypoint, PHP settings, nginx template.

3.2.1 One controller per action, not one per resource

Open `app/Http/Controllers/Api/V1/` and you will find something unusual.

<code>AuthController.php</code>	<code>IncidentSeverityController.php</code>
<code>AuditLogController.php</code>	<code>IncidentStatusController.php</code>
<code>HealthController.php</code>	<code>IncidentUpdateController.php</code>
<code>IncidentAssigneeController.php</code>	<code>MemberController.php</code>
<code>IncidentCommanderController.php</code>	<code>MetricsController.php</code>
<code>IncidentCommentController.php</code>	<code>NotificationController.php</code>
<code>IncidentController.php</code>	<code>OrganizationController.php</code>
<code>IncidentEventController.php</code>	<code>PostmortemController.php</code>
<code>IncidentExportController.php</code>	<code>RealtimeTicketController.php</code>
	<code>ServiceController.php</code>

There are *eight* controllers whose name begins with `Incident`. A more common design would have one `IncidentController` with a dozen methods on it: `updateStatus`, `changeSeverity`, `assign`, `comment`, `export`, and so on.

Splitting them has a specific payoff. Changing an incident's *severity* and changing its *status* need different permissions, emit different timeline events, and notify different people. As methods on one class they would share a file, and over time they would start sharing helper methods and drifting into each other. As separate classes each has exactly one reason to change, and the route file reads as a list of capabilities rather than a list of URLs.

3.2.2 Where the real logic lives

Controllers here are deliberately thin. They check the request, hand off, and format the reply. The thinking happens in `app/Services`.

Service folder	Responsible for
<code>Services/Incidents</code>	Creating and changing incidents; recording the timeline. Three files, and the most important code in the project.
<code>Services/Auth</code>	Issuing and verifying tokens, and the key pair they are signed with.
<code>Services/Notifications</code>	Deciding who hears about what, and writing the rows before anything is sent.
<code>Services/Realtime</code>	Publishing events to Redis so the realtime tier can forward them.
<code>Services/Metrics</code>	Computing time-to-acknowledge and time-to-resolve.
<code>Services/Audit</code>	Writing the append-only audit log.

Careful. If you are ever lost in this codebase, the fastest route is: `routes/api.php` tells you which controller handles a URL; the controller tells you which service does the work; the service is where the answer is. Three hops, every time. Resist the urge to `grep` — following the path teaches you the structure, and `grep` does not.

3.3 Inside web/ — what you look at

Thirty-six TypeScript files, in a conventional React layout.

Folder	What is in it
<code>src/pages</code>	One file per screen. Nine of them — and Part II gives each its own chapter.
<code>src/components</code>	Pieces reused across screens: the layout, the timeline, badges, dialogs.
<code>src/lib</code>	The API client. Every request the browser makes goes through one file.
<code>src/auth</code>	Holding the signed-in user, and refreshing the token before it expires.
<code>src/hooks</code>	Data fetching, and the live connection to the realtime service.
<code>e2e/</code>	Browser tests that drive the whole stack the way a person would.
<code>docker/</code>	The nginx config that serves the built files.

`src/lib/api-client.ts` is worth finding early. *Every* call the frontend makes passes through it, which is why the token refresh, the error shape, and the base URL are each defined exactly once. When you later read that the API returns one consistent error envelope, this is the file that depends on it being true.

3.4 Inside realtime/ — the live connection

The smallest of the three: twelve files, doing one thing — holding connections open and forwarding events onto them.

File or folder	What it does
src/index.ts	Starts the server, and shuts it down carefully.
src/app.ts	The routes: /healthz, /readyz, /stream, /metrics.
src/config.ts	Reads and validates environment variables at boot.
src/ (rest)	Subscribing to Redis, verifying tokens, tracking connections.

Careful. The realtime service can *verify* tokens but cannot *create* them. It is given only the public half of the signing key. This is deliberate: the realtime tier holds thousands of open connections and is the most exposed part of the system, so if it were ever compromised the attacker would gain the ability to read tokens — not to mint new ones. You will see the mechanism in Part II.

3.5 Everything else

Path	What it is
infra/nginx/default.conf	The routing table for the entire system: which URL goes to which service. Read it once and the architecture stops being abstract.
infra/caddy/Caddyfile	The HTTPS terminator, used only when deploying to a single server.
scripts/backup-db.sh	Nightly database backup.
docker-compose.yml	Defines the nine containers, their network, and their storage.
Makefile	Short aliases, so you type <code>make up</code> rather than a 90-character command.

3.6 A rule of thumb for finding anything

You want to change...	Open...
what a screen looks like	web/src/pages/
what data a screen shows	web/src/hooks/
who is allowed to do something	api/app/Policies/
what happens when they do it	api/app/Services/
the shape of the JSON returned	api/app/Http/Resources/
a database column	api/database/migrations/
which URL reaches which code	api/routes/api.php
which service answers a URL	infra/nginx/default.conf

Part II

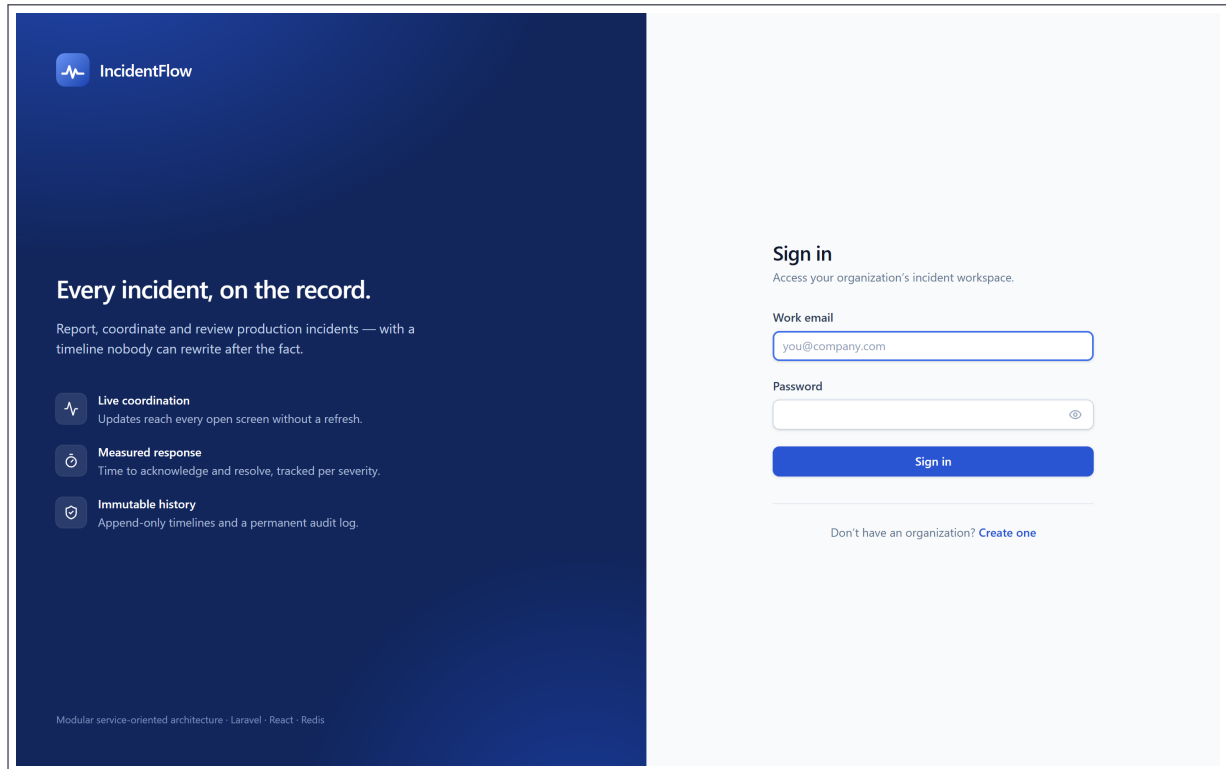
The guided tour

What this part covers

One chapter per screen. Each opens with a photograph of the running application, then shows the code that produced it, then explains the decisions behind that code. The file path is printed above every listing, so you can open the same file on your own machine and read along.

These chapters are meant to be read at the keyboard with the stack running. If it is not running yet, jump forward to Part III, start it, and come back.

Sign in



The sign-in screen. Everything an unauthenticated visitor can see of IncidentFlow.

4.1 What is deliberately missing

Look at the screenshot again and notice what is *not* there: there are no demo account credentials printed on the page.

They used to be. During development it is convenient to show “sign in as `admin@example.com` / password” right on the form. It was removed, and the reason is written into the file so nobody adds it back:

FILE `web/src/pages/LoginPage.tsx`

```
/**
 * Sign-in.
 *
 * Note what is deliberately absent: demo account credentials. They used to be
 * printed here for convenience. A password rendered on a login screen trains
 * every visitor to treat this as a toy, and there is no way for a viewer to
 * tell "seeded demo password" from "real password left in the markup". They now
 * live in the README and the setup manual, where a reader has already chosen to
 * look at project documentation.
 */
```

This is a small thing that says something larger. A visitor cannot tell the difference between a password that is safe to show and one that is not — so showing any password teaches them that this application is careless with them. The credentials did not disappear; they moved somewhere a reader arrives deliberately.

4.2 The two halves of the screen

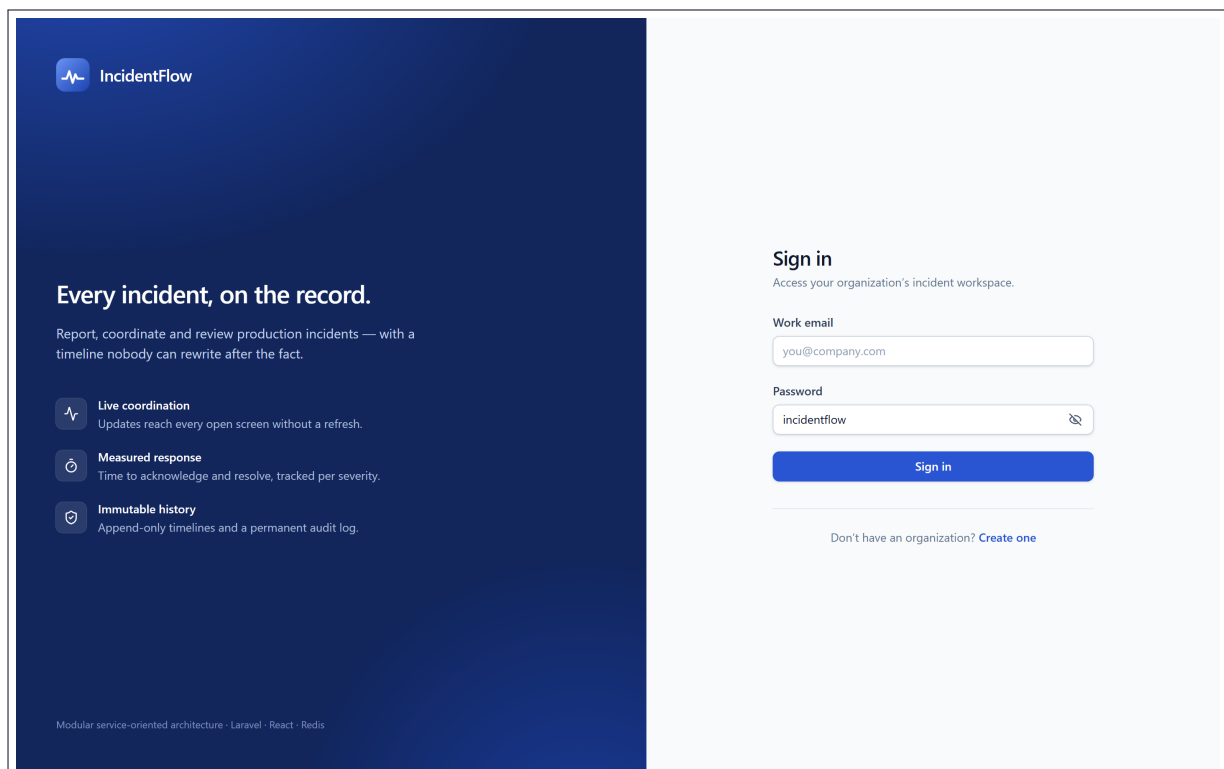
The layout is a split panel: brand on the left, form on the right. The left half vanishes on a narrow screen, and that is a decision rather than a default:

FILE web/src/pages/LoginPage.tsx

```
{/* ----- */}
{/* Brand panel. Hidden below lg: on a phone it would push the actual */}
{/* form below the fold, which is the one thing a login page must not */}
{/* do. */}
{/* ----- */}
<aside className="relative hidden overflow-hidden bg-brand-900 p-12 lg:flex lg:flex-col lg:justify-between">
```

The `hidden` and `lg:flex` classes are Tailwind: hidden by default, becoming a flex column only at the `lg` breakpoint and above. On a phone the form is the first thing on screen.

4.3 The password field, and why it has an eye



The reveal toggle, shown pressed. The field's type flips between `password` and `text`.

FILE web/src/pages/LoginPage.tsx

```

<input
  id="password"
  type={showPassword ? 'text' : 'password'}
  autoComplete="current-password"
  required
  value={password}
  onChange={(event) => setPassword(event.target.value)}
  className="input pr-11"
/>
<button
  type="button"
  onClick={() => setShowPassword((visible) => !visible)}
  // Typos are invisible in a masked field and the error message
  // is deliberately unhelpful, so a reveal toggle is the only
  // way to tell a wrong password from a mistyped one.
  aria-label={showPassword ? 'Hide password' : 'Show password'}
  className="absolute inset-y-0 right-0 grid w-11 place-items-center ..."
>

```

Three details in that block are worth naming.

`type=\{showPassword ? 'text' : 'password'\}`

React re-renders the input with a different `type` attribute. There is no separate “visible” input; it is one field that changes character.

`autoComplete="current-password"`

Tells the browser and password managers what this field is. Without it, saved passwords do not offer themselves, and users blame your application.

`aria-label`

What a screen reader announces for a button whose visible content is an icon with no text.

This broke. That `aria-label` broke every end-to-end test the day it was added. The tests located the password field with `getByLabel('Password')`, and Playwright’s `getByLabel` matches *substrings* — so “Password” also matched “Show password” on the new button. Two elements matched one locator, and Playwright refuses to guess. Full story in Chapter 12.

4.4 What happens when you press Sign in

FILE web/src/pages/LoginPage.tsx

```

async function handleSubmit(event: FormEvent) {
  event.preventDefault();
  setError(null);
  setSubmitting(true);

  try {
    await login(email, password);
  } catch (cause) {
    // The server deliberately returns the same message for an unknown email
    // and a wrong password; passing it straight through keeps the client from
    // reintroducing the enumeration leak the API was careful to avoid.
    setError(
      cause instanceof ApiError ? cause.message : 'Sign-in failed. Check your connection and try
      again.',
    );
  } finally {

```

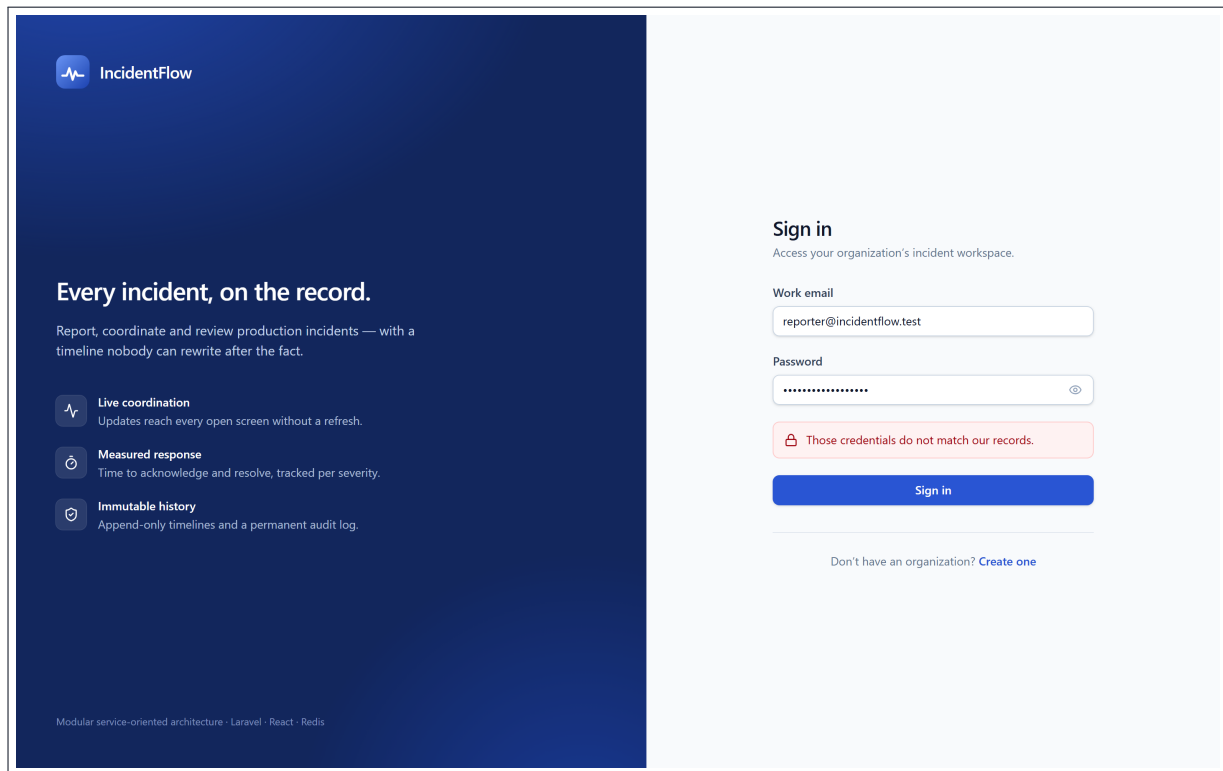
```

    setSubmitting(false);
  }
}

```

`event.preventDefault()` stops the browser’s default behaviour of reloading the page when a form is submitted. Without it, React never sees the submission — the page simply navigates.

4.5 The error message is deliberately unhelpful



A failed sign-in. Read the message carefully.

The message is *“Those credentials do not match our records.”* It does not say whether the email exists.

That is not laziness. Here is the server code:

FILE `api/app/Http/Controllers/Api/V1/AuthController.php`

```

$user = User::query()->where('email', $credentials['email']->first());

/**
 * Hash::check runs even when the user does not exist would be ideal for
 * constant time; Laravel's own attempt() has the same shape. The real
 * defence against enumeration here is the identical response for both
 * cases, plus the per-email+IP rate limit on this route.
 */
if ($user === null || ! Hash::check($credentials['password'], $user->password)) {
    return $this->invalidCredentials($request);
}

```


This defends against **user enumeration**. If a wrong password said “incorrect password” and an unknown address said “no such user”, anyone could discover who has an account by trying addresses — useful for phishing, and for knowing which passwords are worth attacking. One message for both cases removes that signal. The cost is a slightly worse experience for someone who genuinely mistyped their address, which is why the reveal toggle exists.

4.5.1 A disabled account is treated differently

FILE api/app/Http/Controllers/Api/V1/AuthController.php

```
if (! $user->is_active) {
    return response()->json([
        'error' => [
            'code' => 'account_disabled',
            'message' => 'This account has been disabled. Contact an administrator.',
            'request_id' => $request->headers->get('X-Request-Id'),
        ],
    ], Response::HTTP_FORBIDDEN);
}
```

This message *is* specific, and that is consistent rather than contradictory: to reach it you must already have supplied the correct password, so you are not an attacker probing for valid addresses — you are the account holder, and telling you to contact an administrator is the useful answer.

4.5.2 Every error has the same shape

Notice the JSON: code, message, request_id. Every error the API produces has that shape, from any endpoint, formatted in one place.

FILE api/app/Exceptions/ApiExceptionRenderer.php

```
/**
 * One error shape for the entire API.
 *
 * { "error": { "code", "message", "details", "request_id" } }
 *
 * Central error handling is not about tidiness. It is about two properties
 * that are impossible to maintain if each controller formats its own failures:
 *
 * - the frontend has exactly one parser, and switches on a stable `code`
 *   rather than on prose that changes when someone rewords a message;
 * - server faults never leak internals.
 */
```

The request_id matters operationally: it appears in the response *and* in the server log, so a user reporting “it failed” can quote a string that leads straight to the exact log line.

4.6 What sign-in actually returns

Two things, and they are stored differently on purpose.

Token	Lives	Why there
Access token	In memory (JavaScript)	Short-lived, 15 minutes. Sent on every API call. Never written to disk, so a stolen laptop does not hand it over.
Refresh token	httpOnly cookie	30 days. JavaScript cannot read it at all, so a cross-site scripting bug cannot steal it.

You can see the cookie's exact settings by signing in with `curl`:

```
curl -s -i -X POST http://localhost:8080/api/v1/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"responder@incidentflow.test","password":"incidentflow"}' \
  | grep -i set-cookie
```

You should see: A line containing `httponly`, `samesite=lax`, and `path=/api/v1/auth`. That path is a deliberate narrowing: the cookie is only ever sent to the four authentication endpoints, so it is not attached to the hundreds of other API calls that have no use for it.

Careful. In production the same cookie also carries `Secure`, which means the browser will only send it over HTTPS. This has a consequence that has caught many people, including during this project: deploy over plain HTTP and sign-in *appears* to work, then the session dies about fifteen minutes later when the access token expires and the refresh cookie is never sent back. Nothing errors, and no log records it. Chapter 15 covers it.

4.7 Check your understanding

- ☐ Why does the sign-in error not tell you whether the email exists?
- ☐ Why is the access token kept in memory rather than `localStorage`?
- ☐ Why can JavaScript not read the refresh cookie?
- ☐ What would break if `event.preventDefault()` were removed from `handleSubmit`?

The incidents list

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	57m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 30m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

The incidents list, signed in as an incident commander. This is the screen people leave open all day.

5.1 Reading the screen

Seven columns, and each answers a question an on-call engineer actually asks.

Column	The question it answers
Reference	“Which incident are we talking about?” A short stable code you can say out loud on a call.
Severity	“How bad?”
Status	“Is anyone on it?”
Title	“What is it?”
Service	“What is affected?”
Opened	“When did this start?”
Running / resolved in	“How long has this been going on — or how long did it take?”

That last column changes meaning depending on the row. For a live incident it counts up; for a finished one it is a fixed duration. One column, two meanings, and it works because you always want the same thing: elapsed time that matters right now.

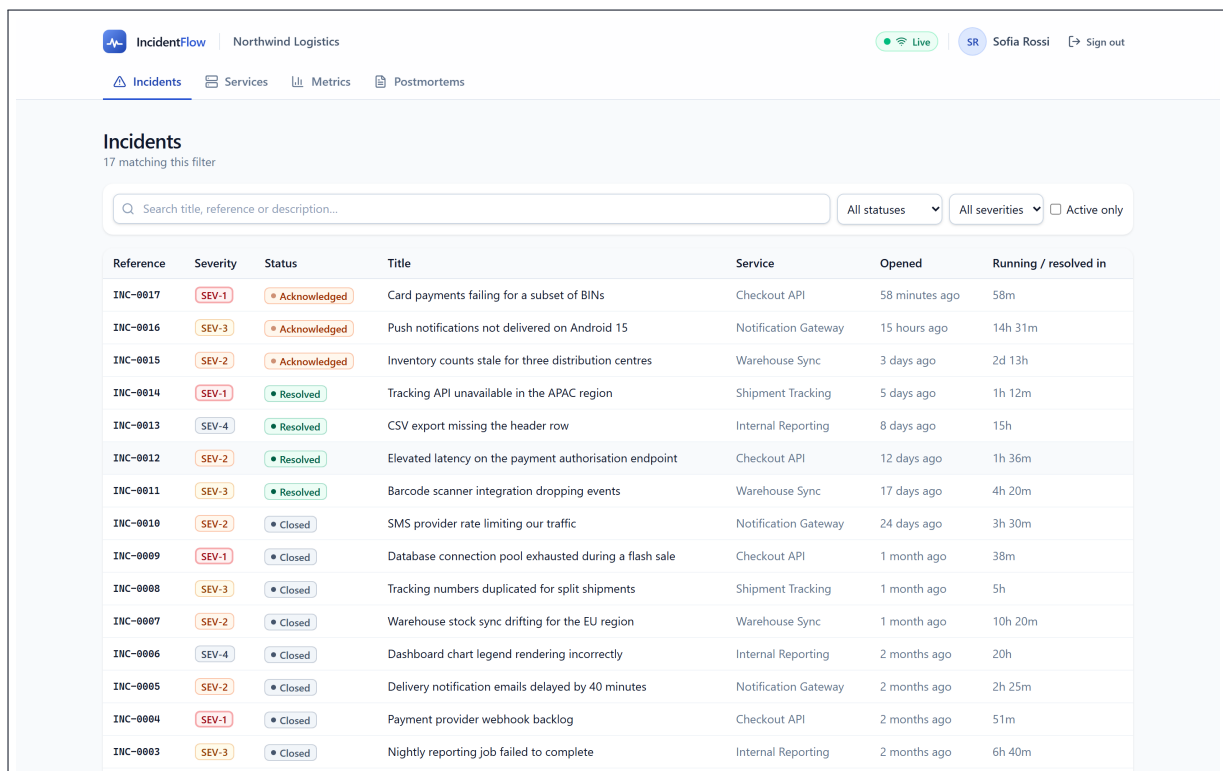
5.2 The button that is not always there

FILE web/src/pages/IncidentsPage.tsx

```
{can('incident.create') && (
  <button
    type="button"
    onClick={() => setCreating(true)}
    className="btn btn-primary"
  >
    <Plus className="h-4 w-4" aria-hidden="true" />
    Report incident
  </button>
)}
```

That `&&` is JavaScript’s short-circuit: if `can('incident.create')` is false, the expression evaluates to false and React renders nothing. The button does not appear disabled — it does not exist.

Compare the same screen seen by a viewer:



The screenshot shows the IncidentFlow application interface. At the top, there's a header with the IncidentFlow logo, 'Northwind Logistics', and user information 'Sofia Rossi' with a 'Sign out' link. Below the header, there's a navigation bar with 'Incidents', 'Services', 'Metrics', and 'Postmortems'. The main content area is titled 'Incidents' and shows '17 matching this filter'. There's a search bar and filters for 'All statuses', 'All severities', and 'Active only'. The table below lists 17 incidents with columns: Reference, Severity, Status, Title, Service, Opened, and Running / resolved in.

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	58m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 31m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

The identical page, signed in as Sofia Rossi, a viewer. The “Report incident” button is gone. So is “Export CSV”.

Careful. This is presentation only. A viewer who opens the browser console and sends `POST /api/v1/incidents` directly is not stopped by a missing button, because the button was never the control. The API refuses the request using a policy on the server. Hiding it is politeness — it keeps people from attempting things they cannot do. Chapter 8 shows both halves side by side.

5.3 Where `can()` comes from

The frontend is told, at sign-in, exactly what this user may do. It does not guess from the role name — it receives a list of permission strings and checks membership. That matters: the mapping from role to permissions lives in the API, in one place, and the frontend never has a second copy of it that could drift.

5.4 Every badge is an enum

FILE web/src/pages/IncidentsPage.tsx

```
<td className="px-3 py-2">
  <SeverityBadge severity={incident.severity.value} label={incident.severity.label} />
</td>
<td className="px-3 py-2">
  <StatusBadge status={incident.status.value} label={incident.status.label} />
</td>
```

Notice the API sends *both* value and label: the machine form (`sev1`) and the human form (Critical). The badge colours itself from value and prints label.

The alternative — sending only `sev1` and having the frontend hold a lookup table of display names — means two copies of the same knowledge. Add a severity level and you must remember to update the other side. Sending both means the API stays the single source of truth for what its own values mean.

5.5 The colour rule this whole design follows

FILE web/src/index.css

```
/* red means severity ... So the brand itself is deliberately not red. */
```

In an incident tool, red already has a job: it means *this is bad*. If the brand colour were also red, every page would be permanently alarming and a real `sev1` would not stand out. So the brand is indigo, and red is reserved for severity alone.

That single rule explains the entire palette — and it is the sort of decision that is obvious in hindsight and invisible unless someone writes it down.

5.6 Filtering, and why the URL changes

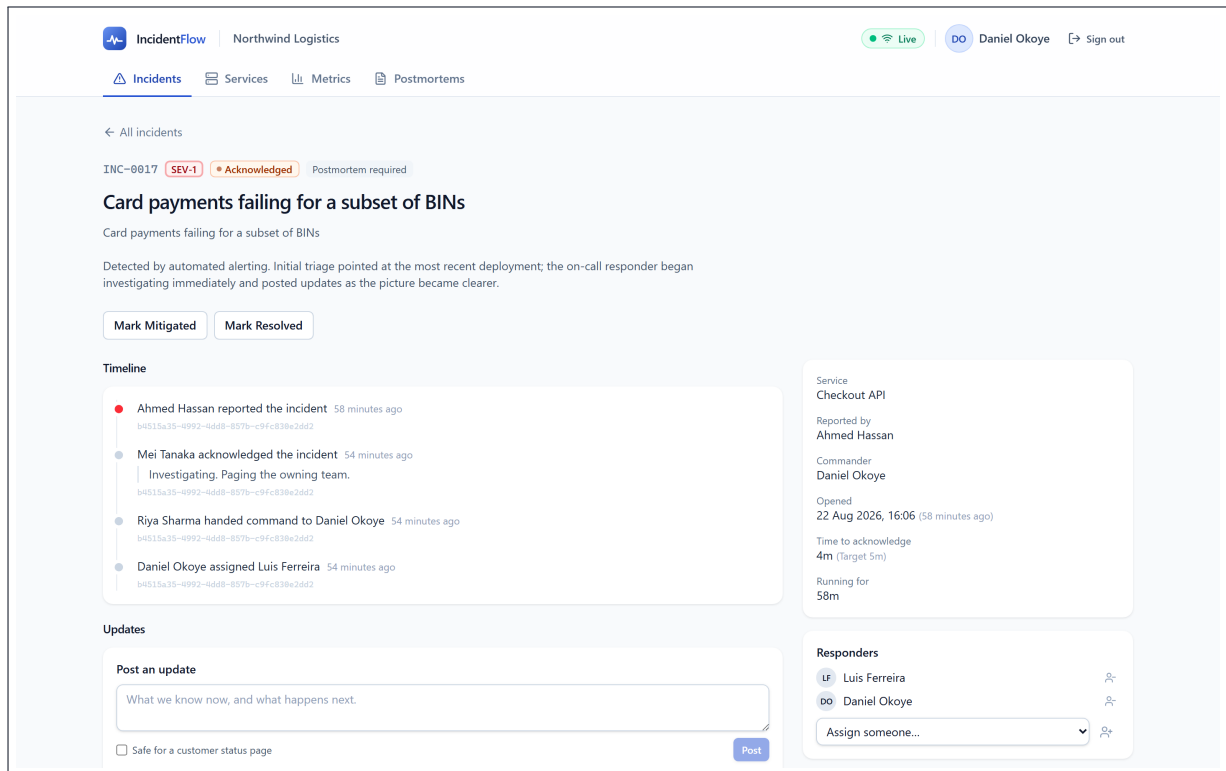
The filter bar above the table — search, severity, status, service — writes its state into the page’s URL. That is why you can copy the address of “all open `sev1` incidents on the payments service” and send it to a colleague, and why the browser Back button steps through your filters.

Storing view state in the URL rather than in a variable is a small amount of extra work that turns a screen into something shareable. On an incident call, pasting a link is faster than describing which dropdowns to set.

5.7 Check your understanding

- ☐ Why does the API send both `value` and `label` for a severity?
- ☐ A viewer cannot see “Report incident”. What actually stops them creating an incident?
- ☐ Why is the brand colour not red?
- ☐ What is gained by putting filter state in the URL?

The incident detail page and its timeline



One incident, open. The timeline runs down the page; the controls for changing it sit alongside.

This is the most important screen in the application. Everything else exists to get you here or to summarise what happened here.

6.1 The timeline is the product

Everything that has ever happened to this incident appears as an entry: it was opened, someone acknowledged it, severity was raised, a note was posted, it was mitigated, it was resolved.

FILE web/src/components/Timeline.tsx

```
{events.map((event) => {
  const { relative, absolute } = formatTimestamp(event.occurred_at);
  return (
    <li key={event.id} className="relative">
      <span className={dotColour(event.type)} />
      <p className="text-sm text-slate-800 dark:text-slate-200">{event.summary}</p>
      <time dateTime={event.occurred_at ?? undefined}>{relative}</time>
      {typeof event.payload.note === 'string' && event.payload.note.length > 0 && (
```

```

        <p>{event.payload.note}</p>
    )}
    {event.request_id && (
        <span>{event.request_id}</span>
    )}
</li>
);
}}}
```

Two details repay attention.

Both time formats

`relative` shows “12 minutes ago”, which is what you want during an incident. `dateTime` carries the exact machine timestamp, which is what you want when writing the postmortem. The `<time>` element is the HTML built for exactly this.

request_id on the entry

Every timeline event records the ID of the HTTP request that created it. When something looks wrong six months later, that string leads to the precise server log line. Very few systems do this, and it costs one column.

6.2 Append-only, enforced three times

Chapter 2 claimed the timeline cannot be edited. Here is the enforcement.

FILE `api/app/Models/IncidentEvent.php`

```

/**
 * Corrections are appended as new events, never applied in place.
 */
protected static function booted(): void
{
    static::updating(static function (): never {
        throw new LogicException('Incident timeline events are append-only and cannot be modified.
    ');
    });

    static::deleting(static function (): never {
        throw new LogicException('Incident timeline events are append-only and cannot be deleted.'
    );
    });
}
```

`booted()` runs once when the class is first used, and registers two listeners. Any attempt to save a change to an existing event, or to delete one, throws — from anywhere in the codebase, including a future controller written by someone who never read this book.

The return type : `never` is doing real work. It tells PHP — and any static analyser — that this function cannot return normally; it always throws. A reader skimming the file learns the guarantee from the signature without reading the body.

6.2.1 Why three layers and not one

Layer	What it stops	What it cannot stop
API	Requests that ask to edit history — there is no such endpoint.	Code inside the application calling the model directly.
Model	Any PHP code in this project, deliberate or accidental.	Someone connecting to PostgreSQL and running UPDATE.
Schema	Structure: there is no “edited_at”, no soft-delete column, nowhere for a revision to live.	A determined administrator with database credentials.

Careful. No layer is complete on its own, and the last row is honest: a person with direct database access can change anything. The guarantee is not mathematical — it is that no *ordinary* path through the system can rewrite history, and that doing so requires deliberate, logged, out-of-band action rather than a stray line of application code.

6.3 Changing an incident

The controls beside the timeline are gated by permission, the same way the “Report incident” button was in Chapter 5. But status changes carry a second gate the list page did not have: even a user *with* permission may only make a *legal* move.

Recall the transition table from Chapter 2: an incident that is **open** may become **acknowledged** or **resolved** — and nothing else. The frontend shows only those two options. The API independently checks the same rule and returns 409 **Conflict** for anything else.

Two independent checks of one rule sounds redundant, and it is — on purpose. They fail differently. The frontend check fails *invisibly*: the option is not offered, so the mistake is never made. The API check fails *loudly*: a 409 with a message. One is for people, the other is for programs, and only the second is a security control.

6.4 The page updates while you watch it

Open this incident in two browsers, change the status in one, and the other updates within a second — no refresh.

That is the **realtime** container. The chain, end to end:

1. The API commits the change to PostgreSQL.
2. After the commit, it publishes a small event to Redis.
3. **realtime** is subscribed to Redis and receives it.
4. It writes the event down every open connection belonging to that organisation.
5. Each browser receives it and updates the page in place.

Careful. Step 4 says “belonging to that organisation” and that phrase is load-bearing. Every connection is authenticated, and events are filtered by organisation before being written. Without that filter, a live stream is a spectacular data leak: every customer would receive every other customer’s incidents in real time, and no page in the UI would ever reveal it.

6.4.1 Why the browser cannot just use its normal token

The realtime connection is opened by the browser’s `EventSource`, which cannot set an `Authorization` header. So the frontend first asks the API for a short-lived *ticket*, then opens the stream with that ticket in the query string.

A ticket is used rather than the access token because query strings are not private: they appear in server logs, proxy logs and browser history. A ticket is valid for seconds, for one connection, and grants nothing but the right to listen. Leaking it in a log is uninteresting; leaking your access token there is not.

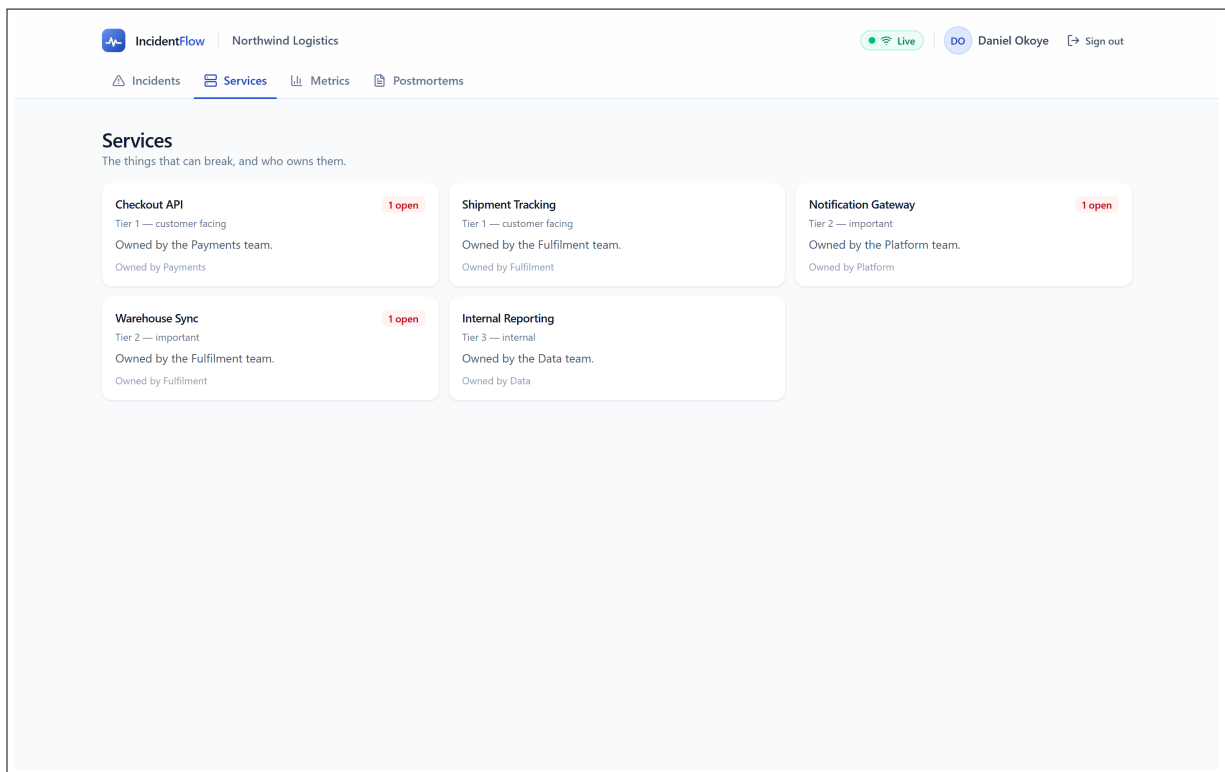
6.5 Check your understanding

- ☐ Name the three layers that stop the timeline being edited, and what each one misses.
- ☐ Why is the same transition rule checked in both the browser and the API?
- ☐ Why does a timeline entry store a `request\_id`?
- ☐ Why does the live stream use a ticket instead of the access token?

Services, metrics and postmortems

Three shorter screens. Each exists because a different person asks a different question after — or before — an incident.

7.1 Services: what can break



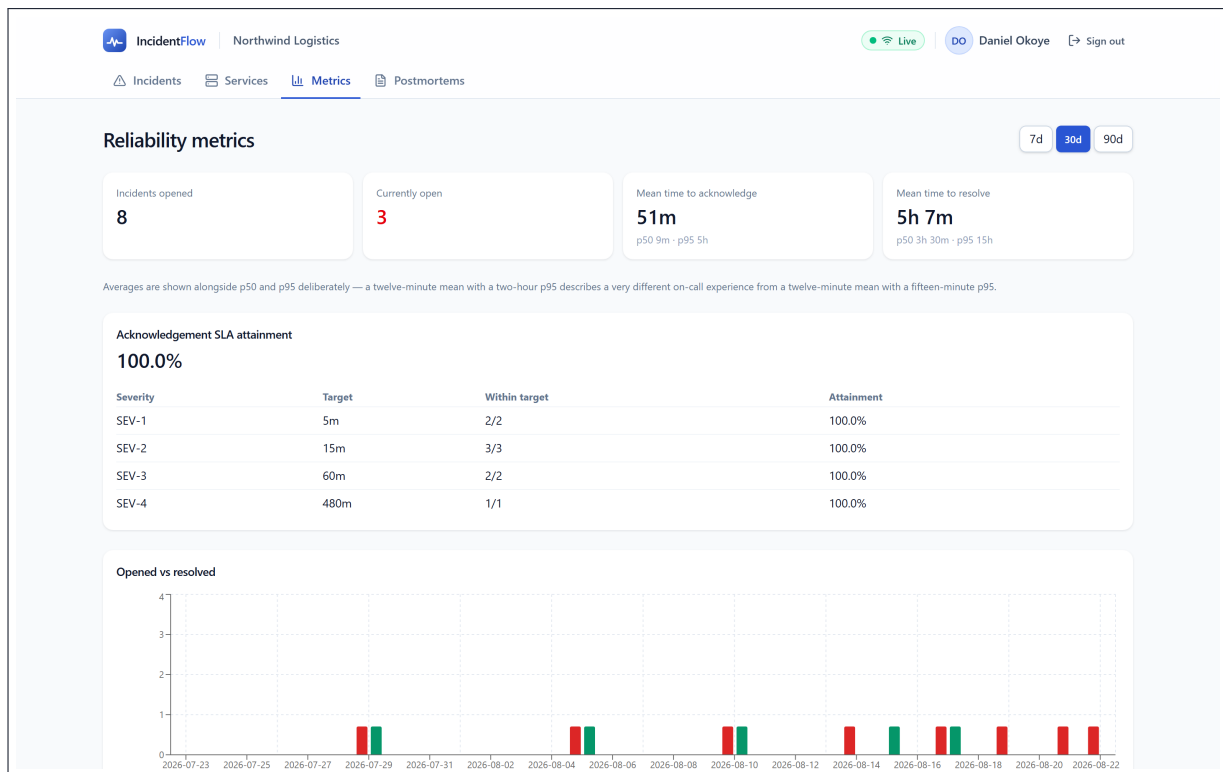
The services registry. Every incident is attached to one of these.

A *service* is a named part of your system: the payments API, the search cluster, the mobile gateway. Incidents attach to one.

This is why the incidents list can be filtered by service, and why the metrics page can say “the payments API had four incidents this month”. Without a registry you would have a free-text field, everyone would spell things differently, and no grouping would ever be reliable. A dropdown of known values is a small constraint that makes every downstream question answerable.

Each service carries an owning team and a criticality. Only an administrator can add or change one — the registry is reference data, and reference data that anyone can edit stops being reference data.

7.2 Metrics: are we getting better or worse?



Reliability metrics over a chosen window.

Two numbers dominate this page, and both are industry standard.

Metric	Full name	What it tells you
MTTA	Mean time to acknowledge	How long before a human picks it up. Measures your <i>alerting and staffing</i> .
MTTR	Mean time to resolve	How long from report to fixed. Measures your <i>systems and tooling</i> .

The distinction matters when deciding what to fix. A bad MTTA and a good MTTR means alerts are not reaching people — a rota problem. A good MTTA and a bad MTTR means people respond promptly and then struggle — a tooling or architecture problem. One number cannot tell you which, which is why there are two.

7.2.1 The numbers are precomputed, not counted on the fly

FILE `api/app/Services/Metrics/`

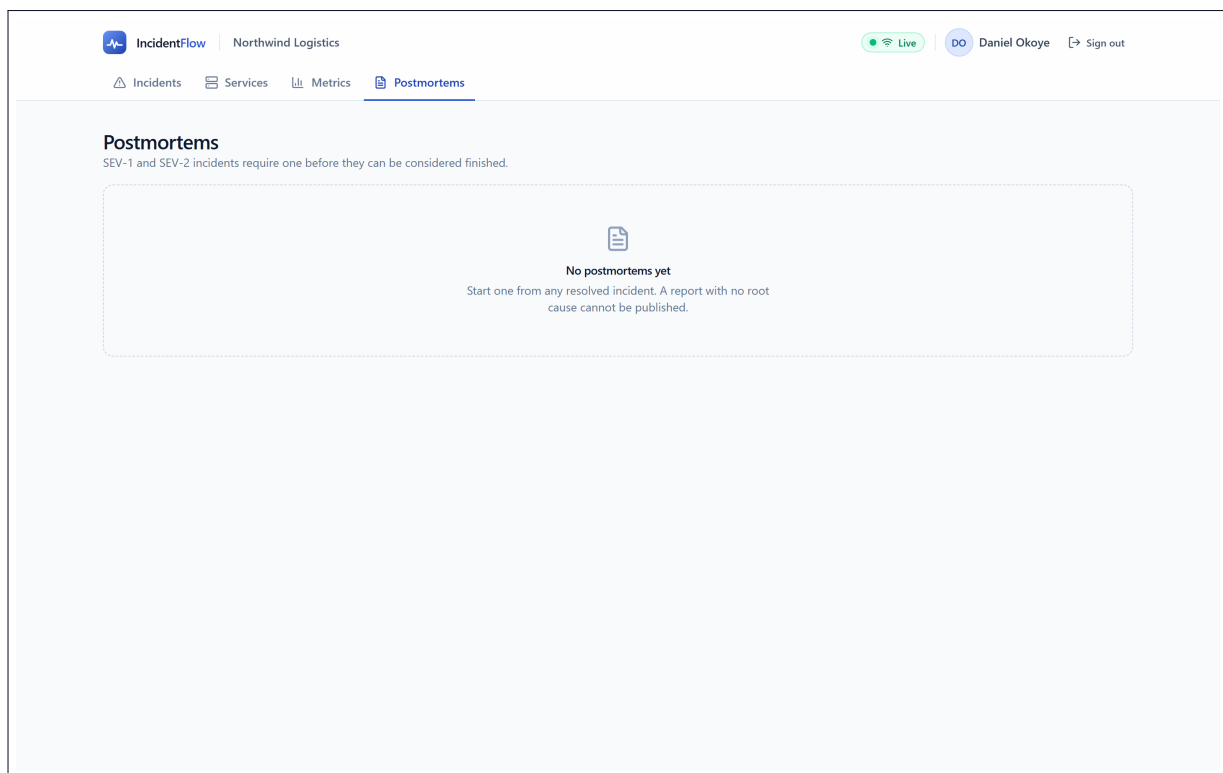
```
/**
 * Reliability metrics: MTTA, MTTR, throughput and acknowledgement-SLA
 * ...
 * columns `time_to_acknowledge_seconds` and `time_to_resolve_seconds` are
 * ...
 * `percentile_cont` nor `date_trunc` exists in SQLite, and a metrics endpoint
 */
```

When an incident is acknowledged, the elapsed seconds are written onto the incident row there and then. The metrics page averages a stored number rather than recomputing durations by scanning the timeline.

Storing a derived value is normally poor practice — it can drift from the truth. It is right here because the value is *immutable once written*: the moment of acknowledgement never changes, so the duration never changes either. The rule of thumb is that caching a derived value is safe exactly when its inputs can no longer move.

Careful. That comment mentions SQLite, and it is a warning from experience. The tests run on SQLite for speed; production runs PostgreSQL. `percentile\_cont` and `date\_trunc` exist only in the latter. Code written naturally against PostgreSQL passes review, fails in the test suite, and — worse — code written to satisfy SQLite can behave differently in production. Chapter 13 contains a bug of exactly this family.

7.3 Postmortems: what did we learn?



Postmortems — the written record produced after an incident is closed.

A postmortem is the document written afterwards: what happened, why, what was done, and what will change so it does not recur.

7.3.1 Postmortems are editable — and that is not a contradiction

The timeline cannot be edited. A postmortem can. That is deliberate and worth being precise about.

	Timeline	Postmortem
What it records	What happened	What we understood
Can change?	Never	Yes, until published
Why	Facts do not change	Understanding improves

FILE `api/app/Models/Postmortem.php`

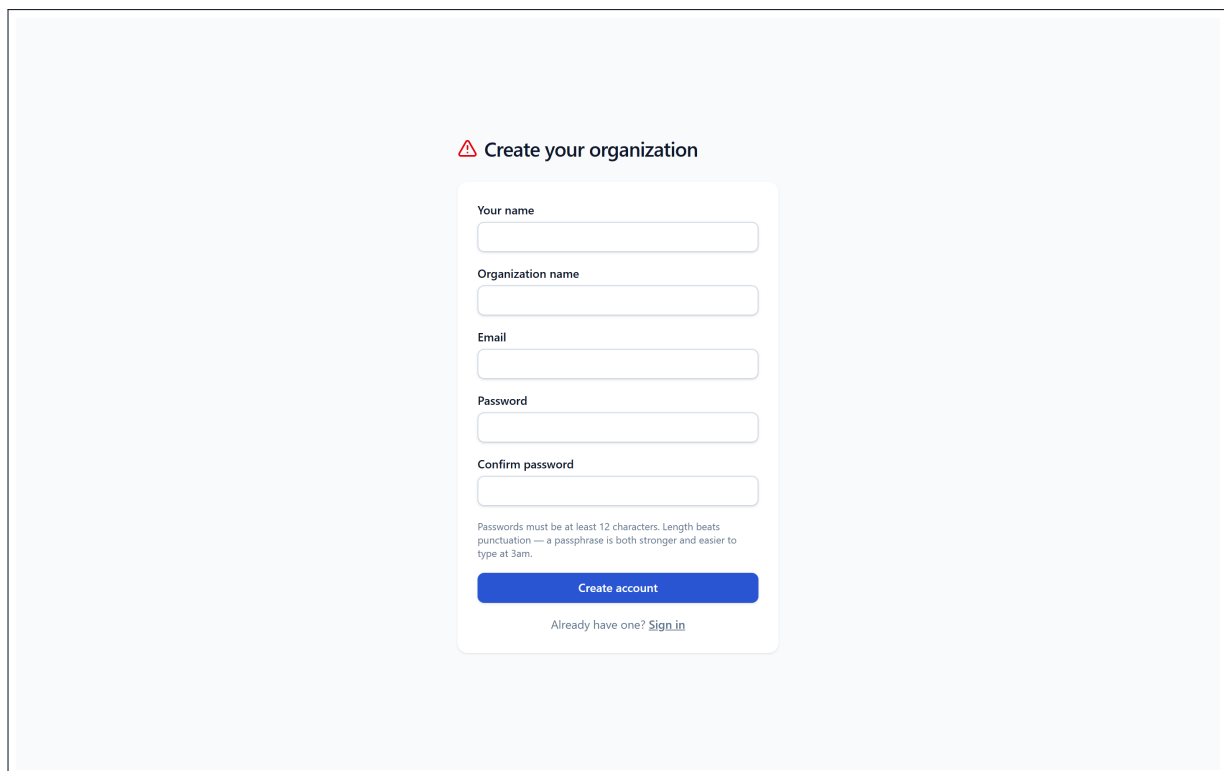
```
public function isPublished(): bool
{
    return $this->status === PostmortemStatus::Published;
}
```

A postmortem is a *draft* while being written, and *published* when finished. Publishing is the point at which it stops being a working document.

This is the same distinction a newspaper makes between a reporter’s notes and a printed article. Conflating them — letting people quietly revise the record of events because their understanding changed — is exactly the failure mode the append-only timeline exists to prevent.

7.4 Two screens you will meet by accident

7.4.1 Registering an organisation



The screenshot shows a web form titled "Create your organization" with a warning icon. The form contains the following fields and elements:

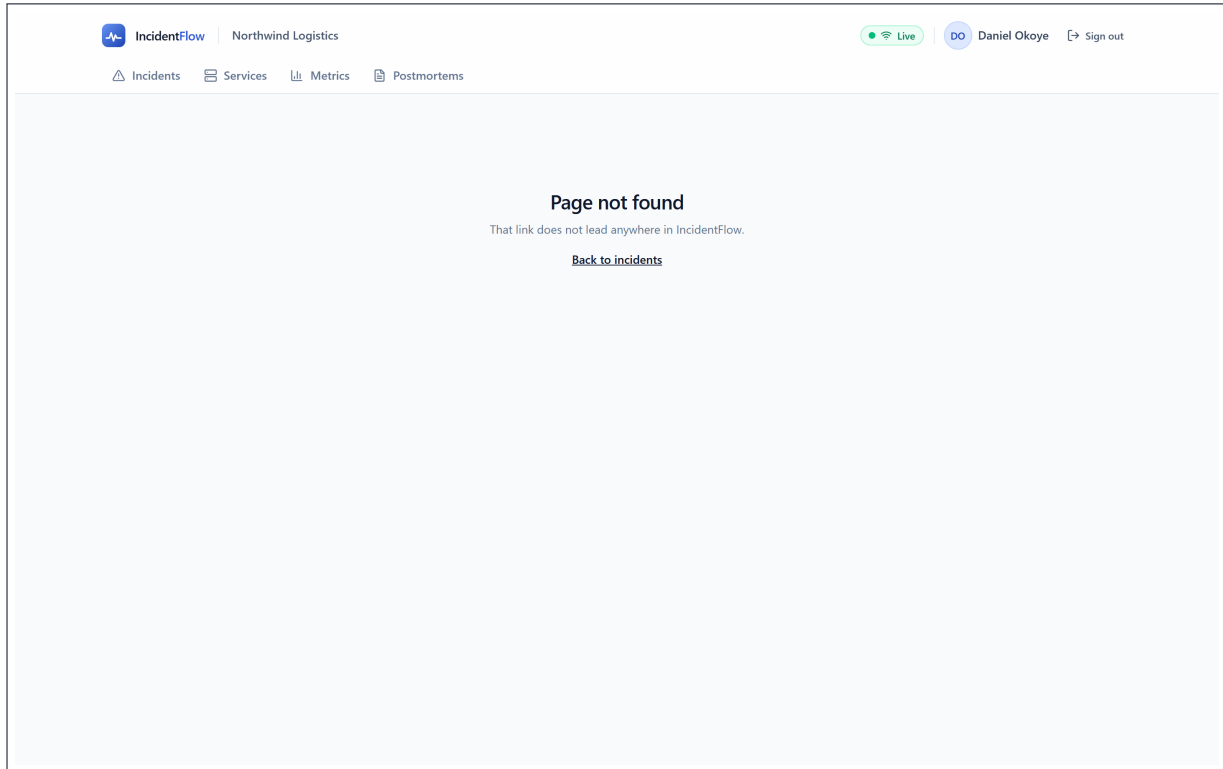
- Your name**: A text input field.
- Organization name**: A text input field.
- Email**: A text input field.
- Password**: A text input field.
- Confirm password**: A text input field.
- Password requirements**: A note stating "Passwords must be at least 12 characters. Length beats punctuation — a passphrase is both stronger and easier to type at 3am."
- Create account**: A blue button.
- Already have one? Sign in**: A link below the button.

Creating an organisation. The first account becomes its owner.

There is no “create an account” for an existing organisation on this screen — registering creates an *organisation* and makes you its administrator. Everyone else is invited from inside.

Careful. This is why a production deployment has no demo logins. The seeder refuses to run outside development, so on a fresh production database the very first thing you do is register, and that first registration becomes the owner. Part III walks through it.

7.4.2 The page that is not there



An unknown URL, inside the application shell.

Notice the navigation is still present. The 404 is rendered *inside* the layout, so a mistyped link leaves you somewhere you can navigate away from, rather than at a dead end.

A single-page application has to be told to do this. The server does not know which paths are real — it hands `index.html` to every unknown URL and lets the router decide. That is the `try_files $uri $uri/ /index.html` line in the web container's nginx config, and without it a hard refresh on `/incidents/42` would return a genuine 404 while clicking through to the same page worked fine.

7.5 Check your understanding

- ☐ Why is a service registry better than a free-text field?
- ☐ What does a good MTTA with a bad MTTR suggest you should fix?
- ☐ Why is it acceptable to store `time_to_resolve_seconds` on the incident row?
- ☐ Why may a postmortem be edited when the timeline may not?

Permissions, seen rather than described

Permission systems are usually taught as a grid, and the grid is forgettable. This chapter teaches it differently: the same page, photographed three times, signed in as three different people.

8.1 One page, three people

IncidentFlow | Northwind Logistics

Live | SR Sofia Rossi | Sign out

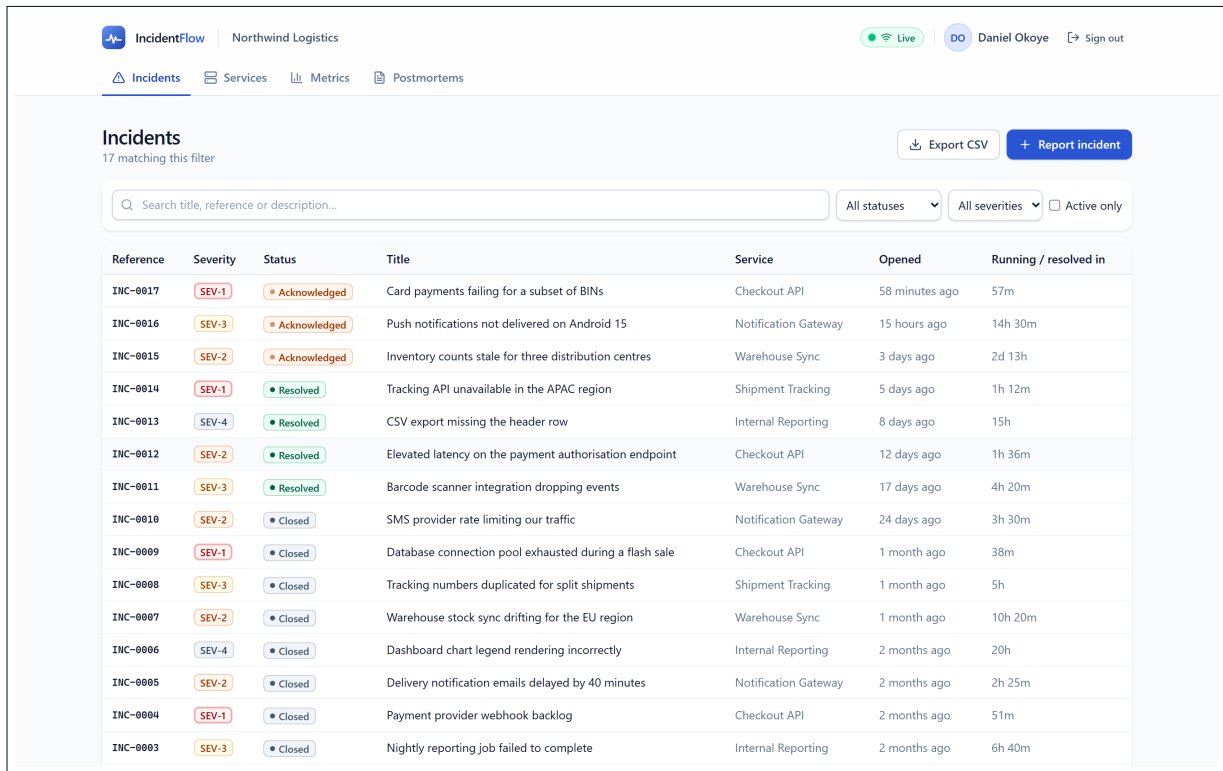
Incidents | Services | Metrics | Postmortems

Incidents
17 matching this filter

Search title, reference or description... All statuses All severities Active only

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	58m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 31m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

Sofia Rossi — viewer. She can read everything and change nothing. No “Report incident”. No “Export CSV”.



IncidentFlow | Northwind Logistics

Incidents | Services | Metrics | Postmortems

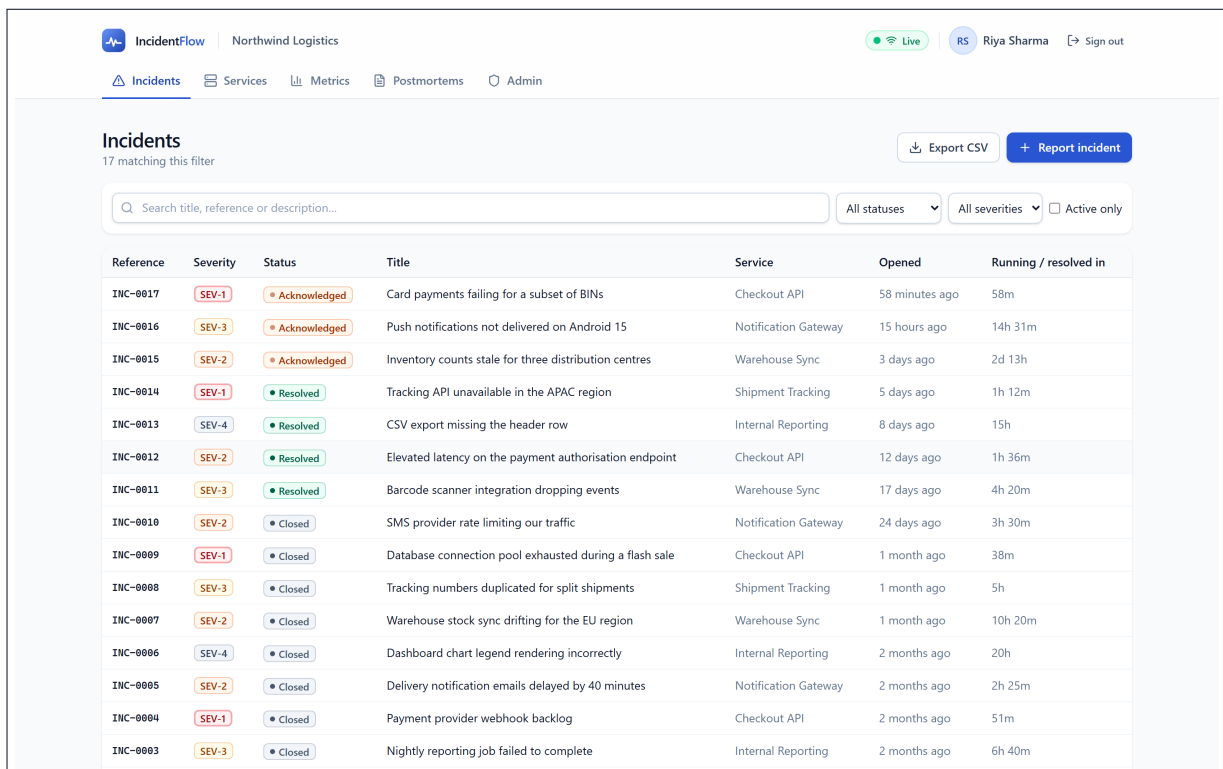
Incidents
17 matching this filter

Export CSV | Report incident

Search title, reference or description... All statuses All severities Active only

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	57m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 30m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

Daniel Okoye — incident commander. The same data, plus the ability to act on it.



IncidentFlow | Northwind Logistics

Incidents | Services | Metrics | Postmortems | Admin

Incidents
17 matching this filter

Export CSV | Report incident

Search title, reference or description... All statuses All severities Active only

Reference	Severity	Status	Title	Service	Opened	Running / resolved in
INC-0017	SEV-1	Acknowledged	Card payments failing for a subset of BINs	Checkout API	58 minutes ago	58m
INC-0016	SEV-3	Acknowledged	Push notifications not delivered on Android 15	Notification Gateway	15 hours ago	14h 31m
INC-0015	SEV-2	Acknowledged	Inventory counts stale for three distribution centres	Warehouse Sync	3 days ago	2d 13h
INC-0014	SEV-1	Resolved	Tracking API unavailable in the APAC region	Shipment Tracking	5 days ago	1h 12m
INC-0013	SEV-4	Resolved	CSV export missing the header row	Internal Reporting	8 days ago	15h
INC-0012	SEV-2	Resolved	Elevated latency on the payment authorisation endpoint	Checkout API	12 days ago	1h 36m
INC-0011	SEV-3	Resolved	Barcode scanner integration dropping events	Warehouse Sync	17 days ago	4h 20m
INC-0010	SEV-2	Closed	SMS provider rate limiting our traffic	Notification Gateway	24 days ago	3h 30m
INC-0009	SEV-1	Closed	Database connection pool exhausted during a flash sale	Checkout API	1 month ago	38m
INC-0008	SEV-3	Closed	Tracking numbers duplicated for split shipments	Shipment Tracking	1 month ago	5h
INC-0007	SEV-2	Closed	Warehouse stock sync drifting for the EU region	Warehouse Sync	1 month ago	10h 20m
INC-0006	SEV-4	Closed	Dashboard chart legend rendering incorrectly	Internal Reporting	2 months ago	20h
INC-0005	SEV-2	Closed	Delivery notification emails delayed by 40 minutes	Notification Gateway	2 months ago	2h 25m
INC-0004	SEV-1	Closed	Payment provider webhook backlog	Checkout API	2 months ago	51m
INC-0003	SEV-3	Closed	Nightly reporting job failed to complete	Internal Reporting	2 months ago	6h 40m

Riya Sharma — administrator. As the commander, plus organisation-level control and an “Admin” item in the navigation.

Nothing about the data changed between those three pictures. What changed is what is *offered*.

8.2 Where the difference is defined

Exactly one file, and it is short enough to read in full.

FILE `api/app/Enums/OrganizationRole.php`

```
public function permissions(): array
{
    $viewer = [
        Permission::IncidentView,
        Permission::PostmortemView,
        Permission::ServiceView,
        Permission::MemberView,
        Permission::MetricsView,
    ];

    $reporter = [
        ...$viewer,
        Permission::IncidentCreate,
        Permission::CommentCreate,
    ];

    $responder = [
        ...$reporter,
        Permission::IncidentUpdate,
        Permission::IncidentTransition,
        Permission::UpdateCreate,
    ];

    $commander = [
        ...$responder,
        Permission::IncidentAssign,
        Permission::IncidentCommand,
        Permission::CommentModerate,
        Permission::PostmortemManage,
        Permission::PostmortemPublish,
        Permission::ExportRun,
    ];

    return match ($this) {
        self::Viewer => $viewer,
        self::Reporter => $reporter,
        self::Responder => $responder,
        self::IncidentCommander => $commander,
        self::Administrator => [
            ...$commander,
            Permission::IncidentDelete,
            Permission::ServiceManage,
            Permission::MemberManage,
            Permission::AuditView,
            Permission::OrganizationManage,
        ],
    };
}
```

8.2.1 Read the shape, not just the contents

The ... is PHP's spread operator: ...\`$viewer` means “everything a viewer can do, and then these as well”. Each role is literally written as the previous one plus a few additions.

This makes the hierarchy *structural* rather than a convention somebody has to maintain. It is impossible to give a reporter a permission a viewer lacks by accident, because a reporter's list is built from the viewer's. A flat table — five arrays written out in full — would allow that mistake, and would allow the five lists to drift apart over years of small edits.

8.2.2 Permissions are an enum, not strings

`Permission::IncidentCreate`, not `'incident.create'`. Typing `Permission::IncidentCreat` is an error your editor catches before you save; typing `'incident.creat'` is a permission check that silently returns false forever — and a silently-false permission check fails *open* or *closed* depending on which side of the comparison it lands, neither of which you will notice in testing.

8.3 Now the same picture from the other side

The screenshots showed the frontend. Here is what actually enforces it:

FILE `api/app/Enums/OrganizationRole.php`

```
public function has(Permission $permission): bool
{
    return in_array($permission, $this->permissions(), strict: true);
}
```

`strict: true` compares type as well as value. Without it PHP's loose comparison can produce surprising equalities, and a permission check is the last place you want a surprise.

Careful. Say this to yourself until it is automatic: **the missing button is not the security**. A viewer who opens the browser console and sends the request by hand meets this check on the server, and is refused with 403. If you ever find yourself relying on a hidden control to prevent something, you have not prevented it.

8.4 The full grid, for reference

Can...	<i>viewer</i>	<i>reporter</i>	<i>responder</i>	<i>commander</i>	<i>admin</i>
view incidents, services, metrics	yes	yes	yes	yes	yes
create an incident	—	yes	yes	yes	yes
comment	—	yes	yes	yes	yes
change status	—	—	yes	yes	yes
post an update	—	—	yes	yes	yes
assign people	—	—	—	yes	yes
change severity	—	—	—	yes	yes
publish a postmortem	—	—	—	yes	yes
export CSV	—	—	—	yes	yes
manage services	—	—	—	—	yes
manage members	—	—	—	—	yes
read the audit log	—	—	—	—	yes

8.5 Try it yourself

With the stack running, sign in as each of these in turn. The password for every demo account is `incidentflow`.

- ☐ `viewer@incidentflow.test` — confirm “Report incident” is absent.
- ☐ `reporter@incidentflow.test` — it appears. Open an incident and confirm you cannot change its status.
- ☐ `responder@incidentflow.test` — now you can.
- ☐ `commander@incidentflow.test` — assignment and severity appear.
- ☐ `admin@incidentflow.test` — an “Admin” item appears in the navigation.

You should see: Five sign-ins, five visibly different versions of the same application. If two of them look identical, you are still signed in as the previous user — sign out properly between them.

The operator surfaces

Three screens that users never see. They are for whoever runs the system — and during development, that is you.

9.1 Admin: the audit log

The screenshot shows the IncidentFlow Admin interface for Northwind Logistics. The top navigation bar includes 'Incidents', 'Services', 'Metrics', 'Postmortems', and 'Admin' (selected). The user 'Riya Sharma' is logged in. The 'Administration' section contains a 'Members' table and an 'Audit log'.

Name	Email	Role	Joined
Riya Sharma	admin@incidentflow.test	Administrator	6 months ago
Daniel Okoye	commander@incidentflow.test	Incident Commander	6 months ago
Mei Tanaka	responder@incidentflow.test	Responder	6 months ago
Luis Ferreira	responder2@incidentflow.test	Responder	6 months ago
Ahmed Hassan	reporter@incidentflow.test	Reporter	6 months ago
Sofia Rossi	viewer@incidentflow.test	Viewer	6 months ago

Audit log
Append-only and permanent. Entries cannot be edited or deleted by anyone, including administrators — an audit log an administrator can rewrite proves nothing about administrators.

- Incident Assigned** by Daniel Okoye 54 minutes ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Commander Changed** by Riya Sharma 54 minutes ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Status Changed** by Mei Tanaka 54 minutes ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Created** by Ahmed Hassan 58 minutes ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Status Changed** by Mei Tanaka 14 hours ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Created** by Ahmed Hassan 15 hours ago
b4515a35-0992-4d08-857b-c9fc838e2dd2
- Incident Assigned** by Daniel Okoye 3 days ago
b4515a35-0992-4d08-857b-c9fc838e2dd2

The administration screen. Members, services, and the audit log.

The audit log is the part worth studying. Every action that changes anything is recorded here: who did it, to what, and when.

This is a *second* append-only record, and it is not the same thing as the incident timeline. The timeline answers “what happened to this incident?” and is visible to everyone. The audit log answers “what did this *person* do?” across the whole organisation, and only an administrator can read it. One is the story of an event; the other is the story of an actor.

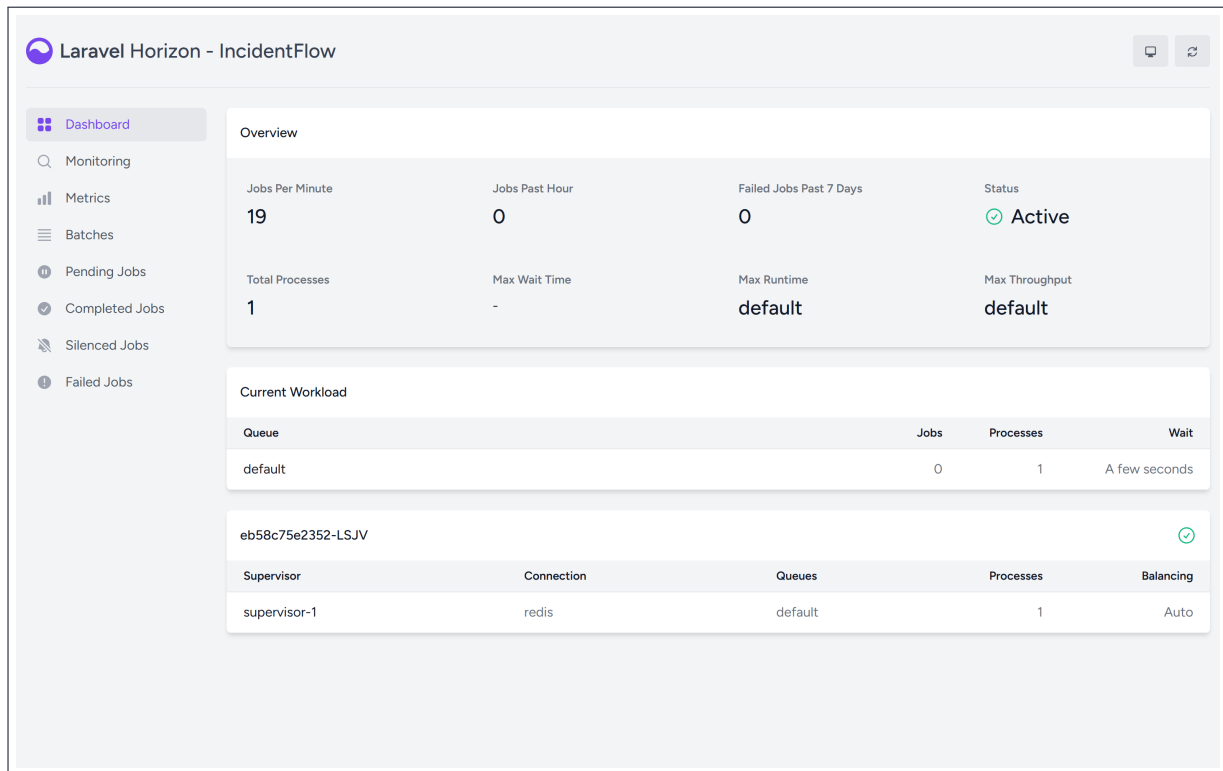
You saw it being written during sign-in, in Chapter 4:

```
FILE api/app/Http/Controllers/Api/V1/AuthController.php
```

```
$this->audit->record('auth.logged_in', $user, $user);
```

Three arguments: what happened, who did it, and what it happened to. For a sign-in the actor and the subject are the same person, which is why `\$user` appears twice.

9.2 Horizon: watching the queue



Laravel Horizon at <http://localhost:8080/horizon>. Jobs processed, jobs failed, throughput.

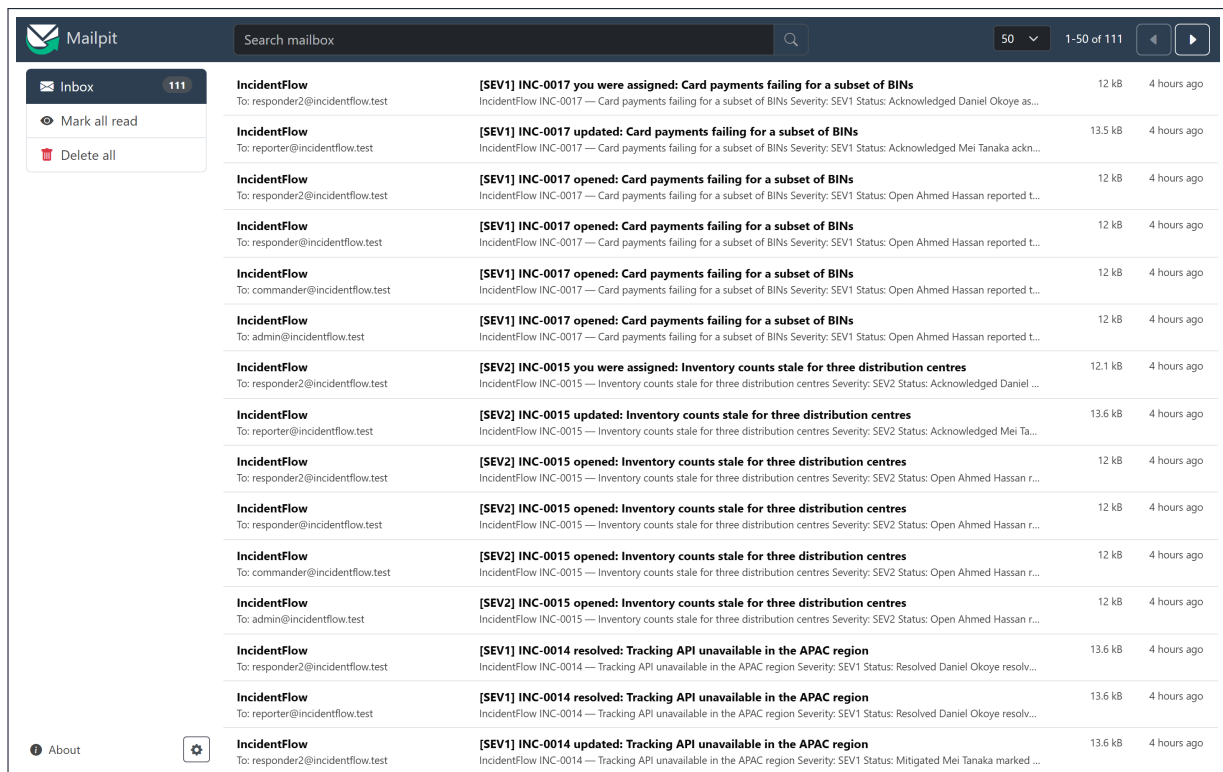
Chapter 2 said the API writes a notification row and returns immediately, leaving `horizon` to do the sending. This is where you watch that happen.

It answers questions that are otherwise invisible:

- Are jobs being processed at all, or is the queue silently stuck?
- Is anything failing repeatedly?
- How long is work waiting before it is picked up?

Careful. This dashboard was completely broken for most of this project's life, and nothing detected it. The page returned HTTP 200. The server logs were clean. The health checks passed. It simply rendered a blank dark rectangle, and the only evidence anywhere was in the browser's developer console. Chapter 14 is that investigation in full — it is the single best example in this book of a failure that no amount of reading the server logs would ever have found.

9.3 Mailpit: catching email that must not escape



Mailpit at <http://localhost:8025>. Every email the system has sent, and not one of them delivered to a real person.

In development, IncidentFlow sends its email to mailpit instead of a real mail server. Mailpit accepts everything and delivers nothing, showing it in a web inbox instead.

This is not merely convenient. The seeded database contains addresses like `commander@incidentflow.test`, but a realistic test dataset eventually contains something that looks like a real address — and a `sev1` notification mails every responder in the organisation. Without a catcher, running the test suite once could send mail to strangers. Mailpit makes that mistake impossible rather than unlikely.

9.3.1 Watch the whole chain in one action

This is the best single exercise in the book, because it exercises five containers at once.

1. Open <http://localhost:8025> in one tab. It should be empty or nearly so.
2. In another tab, sign in as `commander@incidentflow.test` and create an incident with severity `sev1`.
3. Watch the Mailpit tab.

You should see: Email appears in Mailpit within a second or two, one per responder in the organisation. You have just watched: the **api** write rows and commit; **redis** carry the job; **horizon** pick it up; **mailpit** receive the result — and **postgres** holding the record throughout.

Careful. If no mail arrives, the fault is almost always Horizon rather than the mail path. Check `docker compose logs horizon`, and check the Horizon dashboard for failed jobs. A notification row that stays `pending` means the job was never queued or never ran; one marked `failed` means it ran and threw.

9.3.2 Try the same thing at sev3

Create a second incident, this time `sev3`. No email arrives.

That is the rule from Chapter 1: only `sev1` and `sev2` notify the whole responder pool. You have now seen one enum value decide whether a hundred people are interrupted — which is the concrete reason changing severity is a permission most roles do not have.

9.4 Check your understanding

- ☐ What question does the audit log answer that the incident timeline does not?
- ☐ Why does sending email through a queue make the API faster *and* more reliable?
- ☐ Why is a mail catcher a safety feature rather than a convenience?
- ☐ A notification row is stuck at `pending`. What are the two possibilities?

Part III

Running it

What this part covers

Everything needed to take this project from a folder on disk to a working application, assuming you know how to open a folder in VS Code and nothing more. Every command is given in full, with what it does, where you must be standing when you type it, and what you should see afterwards.

It also answers the question people are usually too embarrassed to ask: how many separate things are running, and how do they find each other?

From nothing to running

10.1 What you need installed

Exactly one thing.

Tool	Why
Docker Desktop	Runs all nine containers. Includes <code>docker compose</code> .
Git	To obtain the code.

You do **not** need PHP, Node.js, PostgreSQL or Redis installed. That is the entire point of Docker, and it is worth pausing on: installing PHP 8.3 with the `pdo\_pgsql` extension compiled against PostgreSQL headers is genuinely unpleasant on Windows. Docker does it once, inside an image, on every machine identically.

10.2 The three commands that start everything

Open a terminal. On Windows, press Start, type `powershell`, press Enter.

```
cd D:\IncidentFlow
docker compose up -d --build
```

`up` create and start the containers

`-d` detached — run in the background and give the prompt back

`-build` build the images from source first, rather than reusing whatever was built last

The first run takes several minutes: it downloads base images and compiles PHP extensions. Later runs take seconds.

```
docker compose ps
```

You should see: Nine rows. `postgres`, `redis`, `mailpit`, `api`, `realtime`, `web`, `nginx` should say `healthy`. `horizon` and `scheduler` say only `Up` — they have no HTTP port to check, which is correct and not a fault.

Then load the database:

```
docker compose run --rm migrate
```

You should see: A list of migrations, each ending `DONE`, then seeding output ending with *“Demo accounts seeded. Password for all of them: incidentflow”*.

Now open `http://localhost:8080`.

Careful. If you see the page but signing in fails, the API is not reachable through nginx even though both containers are “up”. Run `curl http://localhost:8080/api/v1/health/ready`. A 502 means nginx cannot reach the API — see Chapter 14, where exactly this happened and the cause was not what it appeared to be.

10.3 Shorter names for the same commands

Makefile at the project root defines aliases. `make up` is `docker compose up -build`, and so on.

Command	What it does
<code>make up</code>	Build and start the whole stack.
<code>make down</code>	Stop everything and remove the volumes .
<code>make logs</code>	Follow every service’s output.
<code>make shell</code>	Open a shell inside the API container.
<code>make migrate</code>	Apply pending migrations.
<code>make seed</code>	Load the demo organisation and 90 days of history.
<code>make fresh</code>	Drop everything and rebuild the schema with demo data.
<code>make test</code>	Run all three test suites.
<code>make lint</code>	Formatting and static checks across all three projects.
<code>make keys</code>	Generate a new RS256 key pair.

Careful. `make down` removes the volumes. That deletes the database — every incident, every timeline entry, every user. It is the right behaviour for a development alias you use to get back to a clean state, and it is a disaster if you type it on a server. Chapter 14 includes the day this distinction stopped being theoretical.

Careful. `make keys` signs every user out immediately, everywhere. The tokens they hold were signed with the old key and will no longer verify. That is correct — it is how you respond to a leaked key — but it is not something to run casually.

10.4 Which command runs where

This trips up nearly everyone at first. There are three different places a command can be typed, and typing one in the wrong place produces confusing errors.

Command starts with	Runs	From where
<code>docker compose ...</code>	on your machine	D:\textbackslash IncidentFlow
<code>docker compose exec api ...</code>	inside the API container	anywhere in the project
<code>npm ...</code>	on your machine	web/ or realtime/
<code>php artisan ...</code>	inside the API container	prefix it with <code>docker compose exec api</code>

`php artisan` is the one that catches people. Typing it directly in PowerShell fails, because PHP is not installed on your machine — it is inside the container. Every `artisan` command in this book is written with its `docker compose exec api` prefix for that reason.

10.5 How the nine containers find each other

Inside Docker, **the service name is the hostname**. When the API connects to the database it uses the host `postgres` — not an IP address, not `localhost`.

FILE `docker-compose.yml`

```
DB_HOST: postgres
REDIS_HOST: redis
```

Docker runs a small DNS server on the container network; `postgres` resolves to whatever address that container currently has.

Careful. “Whatever address it currently has” is the important half. Restart a container and it may get a different one. Anything that resolved the old address and cached it is now talking to nothing — which is precisely how this project’s `nginx` started returning 502 while every container reported healthy. Chapter 14.

10.5.1 The complete connection map

From	To	Port	Carrying
your browser	nginx	8080	everything
nginx	web	80	the React files
nginx	api	9000	FastCGI (not HTTP)
nginx	realtime	3001	the live stream
api	postgres	5432	SQL
api	redis	6379	jobs and events
horizon	redis	6379	taking jobs off the queue
horizon	postgres	5432	reading what to send
horizon	mailpit	1025	SMTP
scheduler	redis	6379	queueing timed work
realtime	redis	6379	subscribing to events

Only the first row crosses the boundary of your machine. Everything else happens on a private network Docker creates, which is why `api` has no published port and cannot be reached from your browser at all.

10.6 Watching it work

```
docker compose logs -f api
docker compose logs -f horizon
docker compose logs --tail 50 nginx
```

`-f` follows — new lines appear as they happen. Press `Ctrl+C` to stop watching; that stops the *watching*, not the container.

10.6.1 Getting inside a container

```
docker compose exec api sh
```

You are now in a shell inside the API container. `php artisan` works here without a prefix, because this is where PHP lives. Type `exit` to leave.

```
docker compose exec postgres psql -U incidentflow -d incidentflow
```

That opens the PostgreSQL client. Try `\textbackslash dt` to list tables, `select count(*) from incidents;` to count rows, and `\textbackslash q` to quit.

You should see: `\textbackslash dt` lists about a dozen tables including `incidents`, `incident\_events`, `users` and `audit\_logs`. The incident count on a freshly seeded database is 17.

10.7 Stopping, and the difference that matters

Command	Effect
<code>docker compose stop</code>	Stops the containers. Data kept. Start again with <code>start</code> .
<code>docker compose down</code>	Stops and removes the containers. Data kept — volumes survive.
<code>docker compose down -v</code>	Stops, removes, and deletes the volumes . Everything is gone.

Careful. That `-v` is the single most dangerous character in this book. On your laptop it means “give me a clean database”. On a server it means “delete the incident history permanently”. There is no confirmation prompt and no undo.

10.8 Running the tests

```
docker compose exec api php vendor/bin/phpunit
```

You should see: OK (114 tests, 547 assertions), in about fifteen seconds.

The browser tests are different — they drive a real browser against the running stack:

```
cd web
npm install
npx playwright install chromium
npm run e2e
```

You should see: 7 passed. These require the stack to be running, because they are not simulating anything — they open a real browser and sign in.

The two suites test genuinely different things. The PHPUnit tests check that each piece is correct in isolation. The Playwright tests check the *seams*: that nginx does not buffer the live stream into silence, that the refresh cookie survives a reload, that the realtime service is subscribed to the channel the publisher actually writes to. Every failure they are designed to catch has the same shape — every service works perfectly on its own and the system still does not.

Part IV

Building it from nothing

What this part covers

The order the pieces have to be built in, and why that order is forced rather than chosen. If you set out to rebuild this project by hand, this is the sequence — and skipping ahead in it produces work you have to throw away.

The order, and why it is not arbitrary

11.1 The principle

Build **inward-out**: the things that cannot change first, the things that change constantly last.

	Stage	Why here
1	The database schema	Everything else refers to it. Getting it wrong later means migrations, data rewrites and edits everywhere.
2	Enums and models	The vocabulary. Once <code>IncidentStatus</code> exists, every later file can use it.
3	Authentication	Nothing else can be built until you can say who is asking.
4	Permissions	Every endpoint needs to ask “may they?” before it exists.
5	Services (the logic)	The actual rules, testable with no HTTP involved.
6	Controllers and routes	A thin shell over the above.
7	The realtime service	Needs events to exist before it can forward them.
8	The frontend	Needs an API to talk to.
9	Docker and nginx	Wires together things that already work individually.
10	CI and deployment	Automates a build you can already do by hand.

The temptation is always to build the screen first, because a screen is visible and a migration is not. It is a false economy: a screen built against an API that does not exist is built against your *guess* at that API, and the guess is usually wrong in ways that are invisible until integration.

11.2 Stage 1: the schema

Eight migrations, and their numeric prefixes are the order they run in.

```
2026_01_01_000100... organizations and users
2026_01_01_000200... membership: who belongs to which organisation
2026_01_01_000300_create_incidents_table.php
2026_01_01_000400_create_incident_activity_tables.php
...
```

Careful. The order is forced by foreign keys. `incidents` has an `organization\id` that must point at a real organisation, so `organizations` has to exist first. Get the numbering wrong and the migration fails — immediately and clearly, which is the good case. The bad case is a schema that lets a row point at nothing.

Design decisions that had to be made at this stage, because changing them later is expensive:

- `incident\_events` has no `updated\_at` and no `deleted\_at`. There is nowhere for an edit to be recorded, because edits are not permitted. The schema states the intent before any code enforces it.
- `time\_to\_acknowledge\_seconds` lives on the incident row rather than being computed from the timeline. Chapter 7 explains why that is safe.
- Idempotency keys have a unique index. That index *is* the mechanism — see Chapter 13.

11.3 Stage 2: enums before models

Write `IncidentStatus` and `IncidentSeverity` before `Incident`. The model then declares its columns *as* those types, and from that moment on it is impossible to assign an invalid status anywhere in the codebase — PHP refuses.

This is why the transition rule lives on the enum rather than in a service. The enum is the one thing every other layer already depends on, so a rule placed there is unavoidable. A rule in a service can be bypassed by not calling that service.

11.4 Stage 3: authentication, and the shape of the key

Authentication had to be built before anything it protects. The decision made here that shaped the rest of the system: **RS256, not HS256**.

	HS256	RS256
Key	One shared secret	A private key and a public key
To verify	You need the secret	You need only the public key
Consequence	Anyone who can verify can also forge	Verifying and signing are separate powers

The realtime service must verify tokens. With HS256 it would need the signing secret, and a compromise of the most exposed service would hand over the ability to mint tokens for anyone. With RS256 it holds only the public half.

Careful. This choice creates an operational requirement that catches people during deployment: *all three* PHP containers must have the same private key. In Docker Compose they share a volume, so it happens automatically. On a platform that gives each container its own filesystem, each would generate a different key and tokens minted by one would fail verification in another. The symptom is intermittent, inexplicable “invalid signature” errors. Chapter 15.

11.5 Stage 5: the logic, before any HTTP

The most important file in the project is `IncidentService`, and it knows nothing about HTTP. It does not see requests or return responses. That is what makes the rules testable without a browser, a server, or a network.

The pattern it follows appears throughout:

1. Check the rules.
2. Open a transaction.

3. Write everything — the change, the timeline entry, the notification rows.
4. Commit.
5. *Then* queue jobs and publish events.

Steps 4 and 5 are in that order for a reason given in Chapter 2, and the project has a dedicated class — `PendingEffects` — whose entire job is to hold the side effects until the transaction has committed. When a codebase grows a class to enforce an ordering, it is usually because getting it wrong once was expensive.

11.6 Stage 9: Docker last, not first

Docker is written when the pieces already work. It is a way of *running* software, not a way of writing it.

File	Written when
<code>api/Dockerfile</code>	The API runs and its tests pass.
<code>web/Dockerfile</code>	The frontend builds and talks to the API.
<code>docker-compose.yml</code>	Every service works individually.
<code>infra/nginx/</code>	There is more than one service to route between.

Careful. Writing the compose file first is a common instinct and a trap: you end up debugging container networking and application logic simultaneously, unable to tell which layer a failure came from. Get each piece working on its own, then wire them together — so that when something breaks, only one thing changed.

11.7 Stage 10: what CI actually protects

The pipeline runs the tests, the linters, a security audit of dependencies, a container vulnerability scan, and then starts the entire stack and runs the browser tests against it.

That last step is the expensive one and the one that has caught the most. Every bug in Chapter 14 lived in the gaps *between* services, where no unit test could reach — and several of them were found only because CI insisted on starting the whole system from nothing, on a machine with no leftover state.

This is the deepest lesson in the book. Local development accumulates state: containers that were already running, a database that was already seeded, a key that was already generated. CI has none of that, every single time. A system that only works on a machine with history is not a working system — it just has not been asked to prove itself yet.

Part V

Bugs, and how they were found

What this part covers

Real failures from building this system. Every one of them actually happened. Each is told in the same four steps: the **symptom**, the **wrong guess** that came first, the **evidence** that corrected it, and the **fix**.

The wrong guesses are included deliberately. Debugging is taught as though the right explanation arrives immediately; it does not. What separates people who are good at it is not better hunches — it is refusing to act on a hunch before checking it.

Bugs in the application

12.1 The button that broke every test

Symptom. A reveal toggle was added to the password field. All seven browser tests failed instantly, with an error about *strict mode*.

Wrong guess. The tests are stale — the page changed, so they need updating. Reasonable, and wrong in an important way: it treats the failure as noise instead of reading it.

Evidence. Playwright’s message named the problem exactly: the locator resolved to *two* elements.

FILE web/e2e/incident-lifecycle.spec.ts

```
await page.getByLabel('Password', { exact: true }).fill(DEMO_PASSWORD);
// Exact, because the field shares the word "password" with the reveal
// toggle's aria-label ("Show password"). getByLabel substring-matches by
// default, so without this the locator resolves to two elements and trips
// strict mode -- which is exactly what happened when the toggle was added.
```

The cause. `getByLabel('Password')` matches *substrings*. The new button’s accessible name is “Show password”, which contains “password”. Two elements, one locator.

The fix. `\{ exact: true \}`.

Strict mode is Playwright refusing to guess. A less careful tool would have picked the first match and passed — and the test would silently have been typing the password into a button from then on. An error that stops you is better than a pass that means nothing, and this is worth remembering when a tool feels pedantic.

12.2 The manual that would have lied

Symptom. The script that captures the screenshots for this book reported success — 21 files written. Four of them were byte-identical to the login page, including the one labelled “the metrics page”.

Wrong guess. A timing problem: the screenshot fired before the page finished rendering. Plausible, and it produced a fix that did not work.

Evidence. Two things, in order. First, file sizes: four files at *exactly* 464 KB, the same as the login screenshot. Identical sizes mean identical images. Second, a trace of the actual network calls, which showed the session being lost partway through the run.

The cause. Two facts that are harmless separately:

- The access token is held in memory only — so a full page reload loses it and the app must call `/auth/refresh` to get another.
- `/auth/refresh` is rate-limited against the *same* per-identity bucket as `login`.

The capture script navigated with `page.goto()`, which is a full reload, so every screen cost one refresh. Add a deliberate failed login for the error-message screenshot — using the same account — and the tour ran out of budget partway through, was logged out, and cheerfully photographed the login page over and over.

The fix. Two changes. Navigate by clicking the sidebar, as a user does, which needs no reloads at all. And spend the deliberate failure on a *different* account.

But the important change was neither of those:

FILE web/capture-screens.mjs

```
if (expect) {
  await page.getByRole('heading', { name: expect }).first()
    .waitFor({ state: 'visible', timeout: 20000 })
    .catch(() => {
      throw new Error(
        `REFUSING to save ${name}.png -- expected heading "${expect}" never appeared. ` +
        `The page is probably showing Sign in instead.`
      );
    });
}
```

Careful. The script now refuses to save a screenshot it cannot verify. This is the general lesson: a process that produces output nobody checks will eventually produce confident, wrong output. A picture is worse than prose here, because a reader trusts a screenshot completely — they assume a photograph cannot be mistaken about what it shows.

Bugs in the backend

13.1 The one that only failed on PostgreSQL

Symptom. All tests passed locally. The same commit failed in CI with `SQLSTATE[25P02]: current transaction is aborted, commands ignored until end of transaction block`.

Wrong guess. A flaky test, or a CI configuration difference. It was neither — it was a real bug that local testing structurally could not find.

Evidence. The tests run against in-memory SQLite for speed; CI also runs them against PostgreSQL. Only PostgreSQL has this behaviour.

The cause. The API prevents duplicate submissions with an *idempotency key*: it inserts a row, and relies on a unique index to reject a second attempt. Catching that rejection is the mechanism.

But in PostgreSQL, **a failed statement poisons the whole transaction**. After the duplicate-key error, every subsequent statement fails with 25P02 until someone rolls back. SQLite does not do this, so locally the code worked perfectly.

The real-world effect: a user who double-clicked “Report incident” got 500 Internal Server Error rather than the polite “you already submitted this” the feature was written to produce.

The fix. Wrap the insert in a nested transaction, which PostgreSQL implements as a *savepoint*. The failure then rolls back only to the savepoint, and the outer transaction survives.

FILE `api/app/Http/Middleware/HandleIdempotency.php`

```
return DB::transaction(fn () : IdempotencyKey => IdempotencyKey::query()->create([...]));
```

One line, and it is not obvious from reading it that it is doing anything. Laravel turns a nested `DB::transaction` into `SAVEPOINT`, so the error is contained. Without the surrounding story, this line looks like it could be deleted.

Careful. The general lesson is the one worth carrying: **a fast local test database that is not the production database will hide a class of bugs, and it will hide precisely the ones that involve transactional behaviour**. The substitute is chosen for speed, and speed usually means fewer guarantees — which is the same thing as fewer failure modes to reproduce.

13.2 The test that proved nothing

Symptom. A newly-added test passed locally and failed in CI — *different* CI job from the last story, and a different cause.

Wrong guess. The rate limit is too low on CI. Checked the logs for 429 responses: none. The guess was wrong, and checking took two minutes.

Evidence. The environment blocks differ:

phpunit.xml	CACHE_STORE = array	fresh for every test
CI, PostgreSQL job	CACHE_STORE = redis	shared for the whole run

The rate limiter keeps its counters in the cache. Nine authentication requests happen in that test file *before* the new test runs. With a per-test cache that does not matter. With a shared one, the per-IP budget was nearly spent and the first attempt in the new test came back 429 — failing an assertion about a different limit entirely.

The fix. Reset the limiter at the start of the test, so it counts from a known state on any driver. Then — and this is the part that mattered — **prove the fix**: run the whole suite against a real Redis, with and without the change, flushing between runs.

with the reset	OK (114 tests, 547 assertions)
without the reset	FAILURES! 1 -- the exact CI failure

Notice the shape repeating from the previous story. Local uses a *more forgiving substitute* — in-memory SQLite, in-memory cache — and both forgive exactly the thing that breaks elsewhere. Test doubles are chosen for speed, and speed almost always means *isolation*; isolation is precisely the property the real component lacks.

13.3 The header that was documented and never sent

Symptom. None. Nothing failed. This was found by writing a test for behaviour nobody had tested.

Evidence. The API’s own error message for rate limiting read: “*Too many requests. Slow down and retry after the period in the Retry-After header.*” The header was not being sent.

The cause. The central error renderer rebuilt every response from the status code and body alone — discarding any headers the exception carried. The throttler attaches `Retry-After` and the `X-RateLimit-*` trio to its exception, and all of them were silently dropped.

The fix.

FILE `api/app/Exceptions/ApiExceptionRenderer.php`

```
/*
 * Carry over headers the exception was throwing *with*, not just its
 * status. The throttler is the case that matters: it raises a 429
 * carrying Retry-After and the X-RateLimit-* trio, and rebuilding the
 * response from scratch silently dropped all of them.
 */
$headers = $e instanceof HttpExceptionInterface ? $e->getHeaders() : [];

return new JsonResponse($payload, $status, $headers);
```

Careful. There was no rate-limiting test at all. The limiters were defined and attached, which *reads* as correct — and nothing proved they refused anything. This is the most common kind of hidden defect in a codebase: not code that is wrong, but code that is never executed by anything that would notice.

13.4 The failure that turned a success into a 500

Symptom. During a CI run with a misconfigured Redis, creating an incident returned 500.

The cause. The incident was written and committed successfully. Then the code tried to queue the notification job, Redis was unreachable, the exception propagated, and the request returned 500.

So the incident *existed* — and the reporter was told it had failed. On a *sev1* at 3am, that person now re-reports it.

The fix. Catch it, log it, and leave the row pending.

FILE `api/app/Services/Notifications/NotificationDispatcher.php`

```
try {
    SendIncidentNotification::dispatch($notification->getKey());
} catch (Throwable $e) {
    /*
     * The queue broker is unreachable. This must not propagate.
     *
     * By the time we get here the incident and its notification rows
     * are already committed -- dispatch() runs after the transaction
     * on purpose. Letting this throw would return 500 for a write
     * that actually succeeded, telling a reporter their SEV-1 was
     * not filed when it was.
     */
    Log::error('notification.enqueue_failed', [...]);
}
```

A scheduled command already existed to sweep notifications stuck in **pending**. The safety net had been built — it was simply unreachable, because the failure it was designed to catch killed the request before the row could be left in the state it looks for. Building a recovery mechanism and then failing loudly instead of entering the state it recovers from is a surprisingly common mistake.

Bugs between the services

These are the hardest failures in the book. In every one, each service was working perfectly and the system still did not.

14.1 The dashboard that returned 200 and showed nothing

This is the best example in the book, because every conventional diagnostic said the system was healthy.

Symptom. `http://localhost:8080/horizon` rendered a blank dark rectangle. No error message on the page.

What all the usual checks said:

Check	Result
HTTP status of <code>/horizon</code>	200 OK
Server error logs	clean
Container health checks	all passing
The HTML shell	delivered correctly
<code>/horizon/api/stats</code>	200, valid JSON

Wrong guess #1. The dashboard’s JavaScript is missing. Checking `/vendor/horizon/app.js` returned 200 — but with `Content-Type: text/html` and a body beginning `<!doctype html>`. That looked like the answer: nginx had no rule for `/vendor/`, so the request fell through to the SPA, which returns `index.html` for unknown paths. The browser asks for JavaScript and receives HTML.

That was a real routing flaw. It was not this bug.

Wrong guess #2. Publish Horizon’s assets to `public/vendor`. Running the command printed: *“Horizon no longer publishes its assets.”* Modern Horizon serves them through its own routes, so `/vendor/horizon/` was never the correct URL to begin with.

The evidence that ended it. Open the browser’s developer console:

```
Executing inline script violates the following Content Security Policy
directive 'script-src 'self''.
```

```
Loading the stylesheet 'https://fonts.bunny.net/css?family=figtree'
violates the following Content Security Policy directive: "style-src 'self'"
```

The cause. The site-wide Content Security Policy is deliberately strict:

FILE `infra/nginx/default.conf`

```
# No CDNs, no external fonts, no inline scripts. The SPA is entirely
# self-hosted, so the policy can be this tight -- and an incident dashboard
# should render even when the wider internet is the thing that is broken.
```


That reasoning is correct *for the React app*. Horizon is a third-party dashboard that boots from an inline `<script>`, loads a webfont from an external host, and — discovered one round later — compiles Vue templates at runtime, which requires `eval`. One policy could not serve both.

14.1.1 The fix that did nothing, and why

The obvious repair is a looser policy scoped to that one location:

```
location /horizon {
    add_header Content-Security-Policy "... 'unsafe-inline' ..." always;
    try_files $uri /index.php?$query_string;
}
```

It had no effect whatsoever. The response still carried the strict policy.

Here is why, and it is a genuinely subtle piece of nginx behaviour. `try_files $uri /index.php` performs an **internal redirect**: nginx stops processing this location and starts again in the location that handles `index.php`. Response headers come from whichever location *finally* handles the request — so the `add_header` in `/horizon` never applied. Worse, `$uri` is rewritten by that redirect and becomes `/index.php`, so testing on `$uri` would not work either. `$request_uri` keeps the original path.

The fix. Choose the policy with a map on `$request_uri`, and emit it once at server level.

FILE `infra/nginx/default.conf`

```
map $request_uri $incidentflow_csp {
    default "default-src 'self'; script-src 'self'; ...";
    ~~/horizon "default-src 'self'; script-src 'self' 'unsafe-inline' 'unsafe-eval'; ...";
}
```

Laravel Horizon - IncidentFlow

Overview

Jobs Per Minute 19	Jobs Past Hour 0	Failed Jobs Past 7 Days 0	Status Active
Total Processes 1	Max Wait Time -	Max Runtime default	Max Throughput default

Current Workload

Queue	Jobs	Processes	Wait
default	0	1	A few seconds

eb58c75e2352-LSJV

Supervisor	Connection	Queues	Processes	Balancing
supervisor-1	redis	default	1	Auto

Horizon after the fix. “Jobs Per Minute”, “Failed Jobs Past 7 Days” — the queue is finally visible.

Careful. The relaxation is real, and the config says so rather than pretending otherwise. `unsafe-inline` plus `unsafe-eval` is a meaningful weakening. It is accepted because it is scoped to an authenticated operator path, it cannot reach the application, and the alternative was having no queue visibility at all. Write down what a compromise costs; a comment that only records the benefit is marketing.

14.1.2 The lesson

Every server-side signal said healthy. The only evidence existed in the browser, in a console nobody had opened. **A class of failure exists that your server can neither detect nor log**, because the rejection happens on the other machine — and a policy failure is exactly that class: nothing is broken, everything behaves as specified, and the specification forbids the thing you needed.

14.2 The 502 that appeared while nothing changed

Symptom. The application had been working. Later, every API request through nginx returned 502 Bad Gateway. No deploy, no code change.

Wrong guess. PHP-FPM has died or is listening on the wrong interface. Checking inside the container showed it listening on `:::9000` and alive.

Evidence. The container uptimes:

```
nginx    Up 4 hours
api      Up 3 hours
```

The API restarted *after* nginx started.

The cause. nginx resolves the hostnames in an `upstream` block **once, at startup**, and caches the address. The API container restarted and Docker gave it a new IP. nginx was still dialling the old one.

The fix. `docker compose restart nginx` — and understanding that this will recur every time the API restarts alone.

The same class of bug appears in the deployment configuration, where upstreams are declared with a resolver and re-resolved at request time specifically so a redeploy of one service does not leave the edge pointing at a dead address. Recognising a bug you have already met, in a different disguise, is most of what experience is.

14.3 The health check that lied

Symptom. CI failed intermittently: `migrate` didn't complete successfully: `exit 1`, while PostgreSQL reported **healthy**. Roughly one run in two.

Evidence. Sampling both forms of the readiness probe on a fresh database, every 0.3 seconds:

```
t=00.0s  socket(no -h): --      tcp(-h 127.0.0.1): --
t=01.8s  socket(no -h): UP      tcp(-h 127.0.0.1): --      <-- healthy, nothing listening
t=02.1s  socket(no -h): --      tcp(-h 127.0.0.1): --
t=03.3s  socket(no -h): UP      tcp(-h 127.0.0.1): UP
```

The cause. The check was `pg\_isready -U incidentflow -d incidentflow`. With no `-h`, `pg\_isready` connects over the **Unix socket**. The official Postgres image runs its initialisation phase on a temporary server started with `listen\_addresses=""` — socket only, no TCP. So the probe succeeds

mid-initialisation, the container is marked healthy, and everything waiting on it is released against a port that accepts nothing.

There was even a comment claiming the check tested a real connection by naming the user and database. It does not: `pg\_isready` never opens a database connection, it asks the postmaster whether it is accepting them.

The fix. `pg\_isready -h 127.0.0.1 -U incidentflow -d incidentflow`.

Careful. The general rule: **a readiness probe must use the same transport as the client it gates**. Probing a Unix socket to certify TCP readiness tests a different thing from the one that matters — and the two diverge exactly during startup, which is the only window the probe exists for.

14.4 Three smaller ones, quickly

14.4.1 A clean exit treated as a failure

`docker compose up -wait` returned 1 with `migrate Exited (0)`. The migration container does its job and stops, and `-wait` counted a stopped container as a failure. The fix is `condition: service\_completed\_successfully`, which also made `api`, `horizon` and `scheduler` wait for the schema to exist — something they had never actually been told to do.

14.4.2 Workers permanently unhealthy

`horizon` and `scheduler` inherited the API image's health check, which probes `php-fpm` on port 9000. Neither runs `php-fpm`. Both sat marked unhealthy while working perfectly — Horizon was visibly processing jobs while reporting itself broken. Disabling an inherited check that describes a different process is the fix.

14.4.3 The second ‘up’ that destroyed the stack

Running `docker compose up` a second time, without removing volumes, aborted everything: the seeder re-inserted blindly and hit a unique constraint, and because the other services had (correctly) been made to wait for the migration to *succeed*, its failure took the whole stack down.

Demo data already existing is the normal state on a re-run, not an error:

FILE `api/database/seeder/DatabaseSeeder.php`

```
if (Organization::query()->exists()) {
    $this->command?->info('Demo data is already present; nothing to seed.');
```

```
    return;
}
```

Notice the interaction. Making dependencies stricter was correct, and it converted a previously harmless failure into a total outage. Tightening one part of a system can promote an existing minor fault into a major one — which is an argument for tightening things, not against it: the fault was always there.

Bugs that only appear when deploying

15.1 The cookie that is never sent

Symptom (predicted, not suffered). Deployed over plain HTTP, sign-in appears to work and the session dies roughly fifteen minutes later. Nothing errors. Nothing is logged.

The cause. The production configuration sets `JWT_REFRESH_COOKIE_SECURE=true`. A Secure cookie is only ever sent back over HTTPS. Over plain HTTP the browser accepts it and then never returns it — so the first refresh, when the fifteen-minute access token expires, has no credential.

Every component behaves exactly as specified. The API sets the cookie correctly. The browser stores it correctly. It is simply never transmitted.

Careful. This is why the deployment guide treats TLS as mandatory rather than a finishing touch. “It works on HTTP, I will add HTTPS later” produces an application that appears to work and logs people out fifteen minutes later, in a different component, with no error anywhere.

15.2 Three containers, three different signing keys

The setting. Moving from Docker Compose to a platform that gives each container its own filesystem.

The cause. In Compose, `api`, `horizon` and `scheduler` share one volume holding the JWT key pair. On a platform without shared volumes, each container generates its *own* pair at startup. A token signed by one then fails verification in another.

The symptom would be intermittent, inexplicable “invalid signature” errors depending on which container answered.

The fix. Inject the key as an environment variable rather than generating it. The configuration already supported this:

FILE `api/config/jwt.php`

```
'private_key' => env('JWT_PRIVATE_KEY'),
'private_key_path' => env('JWT_PRIVATE_KEY_PATH', storage_path('keys/jwt-private.pem')),
```

This is the shape of nearly every compose-to-platform port. The bug is not in your code — the code is correct. It is that a *shared volume* is an ambient guarantee Compose provides in one line and the platform silently removes. The same is true of a flat IPv4 network and of `depends_on` ordering. Each has to become explicit configuration or it becomes a runtime mystery.

15.3 A hostname that resolves and still cannot be reached

The cause. One platform's private DNS resolves service names to **IPv6 only**. The API's nginx listened with `listen ${PORT};`, which binds IPv4 alone. The hostname resolved perfectly; nothing could connect.

The fix. Bind both stacks.

FILE `api/docker/render-nginx.conf.template`

```
listen ${PORT};
# Dual-stack. Railway's private DNS (*.railway.internal) resolves to IPv6
# only, so an IPv4-only listener is unreachable from a sibling service
# however correct the hostname is. Harmless on Render, required here.
listen [::]:${PORT};
```

Careful. Verifying this needed care. `nginx -t` only checks *syntax* — a configuration that cannot bind still passes. The check that matters is starting it and confirming both sockets are actually listening and serving. A test that validates a file is not a test that validates a program.

15.4 A message that was a count, not a diagnosis

Symptom. A container refused to start, reporting: *database unreachable after 30 attempts*.

That sentence contains no information. It cannot distinguish an unresolvable hostname from a refused connection from a rejected password — three completely different problems with three completely different fixes. The probe had the real error and threw it away:

```
until php -r "new PDO(...);" 2>/dev/null; do
```

That `2>/dev/null` discarded the driver's message on every one of the thirty attempts.

The fix. Keep the last error and report it.

```
until db_error="$(php -r "new PDO(...);" 2>&1)"; do
...
# Report what the driver actually said. "unreachable after 30 attempts" is
# a count, not a diagnosis.
echo "{\"level\":\"fatal\",\"message\":\"database unreachable\",\"last_error\":\"${detail}\"}"
>&2
```

The two failure modes it had been conflating, now distinguishable:

```
could not translate host name "nosuchhost" to address: Name does not resolve
connection to server at "postgres" failed: FATAL: password authentication failed
```

Suppressing `stderr` in a retry loop is extremely common — it keeps the logs tidy while the loop retries. It also means that when the loop finally gives up, the one moment you need the reason, it has been discarded thirty times over. Keep the last error even when you discard the rest.

15.5 Two mistakes of mine, for completeness

15.5.1 The tool that reported success without running

A formatting check reported “passed” every single time, locally — because a hook intercepted the command and returned a canned success. Every local “lint is clean” was meaningless, and CI kept failing on formatting.

Careful. The lesson generalises well beyond this project: **a check that cannot fail is not a check.** If you have never seen a tool report a failure, you do not know it is running. Break something on purpose and confirm it complains.

15.5.2 The document that destroyed itself

The first attempt at writing this book generated the LaTeX through a shell heredoc. Every `\textbackslash \textbackslash` collapsed into a single backslash. LaTeX builds table rows and line breaks out of `\textbackslash \textbackslash`, so every table in the document was silently destroyed — and the error it produced pointed at a title line that looked perfectly correct.

Both of these share a shape with the screenshot bug in Chapter 12: a tool reported success while producing wrong output. That is the failure mode worth fearing most, because every instinct you have says to trust it.